LECTURE 07

需求和架构（2）

07

设计可生长的解空间：从共同接口到可重算的结果

25 节 · 46 幅配图 · 9 道自测　回看课堂 ↗

完整阅读复用导出的20个转写分块（909句，00:00.96—1:39:41.04）；未逐秒听审，原音频/视频末尾已解码检查。

官方讲义39页完整读取并截图，图中包含当前官网补充内容；正文明确区分课堂原意、讲义补充与整理者解释。未宣称读完讲义所引全部经典原文。

讲义图片来自当前官方网页，汇编PDF为本次截图生成；其页码与视频内页码/原书印刷页码分开，相关时间只作讲解导航。

视频动态演示按重点采样，保留AI导读、文件选择到nano打开、记账回放板书等；原媒体1280×410，细字清晰度有限。

BaoCut显示ASR stale、asrLayout与polish fresh，transcribe任务done；核对视频身份、首尾、媒体SHA256并复用导出，不将该状态称作fresh ASR。

桌面和窄屏浏览器验收由批次协调者补充，未做真实手机设备测试。

阅读导引

先把握本课主线

01 本讲从需求与架构的取舍出发，借结构化程序设计与Growing a Language说明：架构要尊重有限注意力，为变化提供**可组合的空间**。

02 UNIX、关系数据库、Internet的共同之处，是**固定协作规则**，让使用者与下层实现能持续演化。

03 MVC与前端状态模型使变化各有归属；长业务流程仍需把过去的事实、现在的视图、**未来的预期**分开。

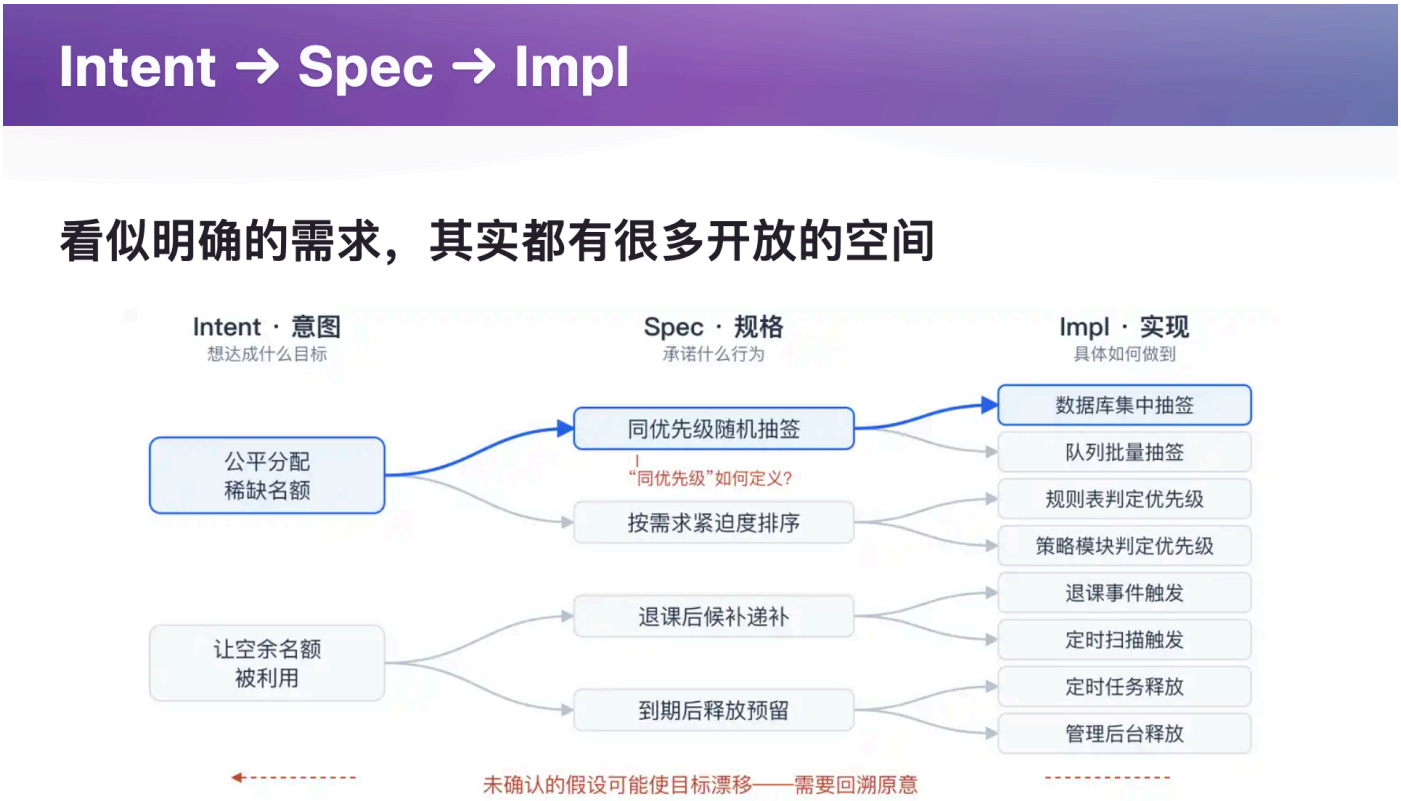
04 记账与成绩更正案例落到一个检查：重要结果能否说明依据什么、怎样算出、**输入变化后如何重算**。

第 01 节 00:00:00—00:03:00 · 课堂时间

从能运行到能长期维护：需求与架构之间的空隙

> **关键判断 · 整理者提炼**
>
> 运行成功只验证当前交付；架构还要比较数据表示和边界对未来维护的影响。

本讲延续上一讲：Intent（意图）、Specification（规约）与 Implementation（实现）之间并非一一对应。即使把教师端教务系统逆向成了一份完整需求清单，照单实现也只证明当前功能可交付，不能证明**未来的维护代价**合理。选择数据库、数据表示和**模块边界**的那一刻，就已经影响了以后的道路。



资料图 · 第 1 页 官方网页讲义第1/39页：Intent → Spec → Impl。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 1 · 原始讲义 · 相关讲解 00:00:00（不是同秒视频截图） · 官方网页讲义第1/39页：Intent → Spec → Impl。相关时间仅定位本节讲解，非同秒视频截图。

架构不是大项目的专属名词。十行汉诺塔、两万行二维码工具和几十万行教务系统，都包含“用什么概念表示问题、让哪些部分互相知道”的选择。代码规模越大，早期决定传播的影响通常越难回收。

资料图 · 第 2 页 官方网页讲义第2/39页：架构无处不在。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 2 · 原始讲义 · 相关讲解 00:00:00（不是同秒视频截图） · 官方网页讲义第2/39页：架构无处不在。相关时间仅定位本节讲解，非同秒视频截图。

讲师提出从**代码外部**观察问题：客户的工作规律、社会的规则、数学与物理规律，都可能提供比现有页面或代码更合适的组织方式。第一次想到一个能工作的方案后，值得再退一步比较其他可能，而不是马上把它当成唯一答案。

> 案例回扣 实验建议：可以先把需求交给 AI，获得可运行版本，再通过对话修改；但复盘时应另问“如果重新设计数据和边界，哪些后续修改会更容易？”**运行成功与架构质量**是两次不同的检查。

第 02 节

00:03:01—00:05:44 · 课堂时间

本质复杂性与附带复杂性：先判断困难从哪里来

> **关键判断 · 整理者提炼**
>
> 领域约束不会因换工具消失，实现选择则会增加不同的同步、接口和维护代价。

Essential Complexity（**本质复杂性**）来自问题领域：课程有容量，学生有先修约束，课程时间冲突，教室还可能被借用。即使没有软件、由人拿纸笔办理，这些约束也不会消失。架构要帮助我们管理它们，不能靠换语言把它们消灭。

Accidental Complexity（**附带复杂性**）来自实现选择：文件还是数据库、数据重复保存几份、格式是否一致、同步如何完成。不同选择会形成不同的**取舍空间**，所谓“简单”必须说明对哪一部分简单。

资料图 · 第 3 页 官方网页讲义第3/39页：软件工程的本质：驾驭复杂性。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 3 · 原始讲义 · 相关讲解 00:03:01（不是同秒视频截图） · 官方网页讲义第3/39页：软件工程的本质：驾驭复杂性。相关时间仅定位本节讲解，非同秒视频截图。

课堂比较了文件系统与数据库：文件便于直接查看、修改，并复用 UNIX 工具；数据库通过事务等机制封装并发与一致性困难，但查询与调试多了一层接口。没有任何选择能同时消除全部代价。

> 案例回扣 多文件读写不能仅凭普通文件操作就获得数据库式的 all-or-nothing 事务。课堂以“仿佛锁住整个世界”帮助理解原子性；这是直觉比喻，不表示所有事务都实际停住全系统。

> **整理者说明**
> 整理者边界：此处讨论普通文件操作与事务数据库的典型差别，不是断言世界上不存在事务文件系统，也不把数据库可靠性当成无条件保证。

第 03 节

00:05:44—00:10:50 · 课堂时间

一个登录需求怎样展开成产品决策树

> **关键判断 · 整理者提炼**
>
> 一句“支持登录”会展开成持续维护的产品承诺，关键决策应比较替代方案。

需求工程直接面对本质复杂性，通过沟通减少“客户到底要什么”的不确定性。讲师推荐尝试**持续追问细节**的需求澄清方式：不能仅写“支持登录”，就任由模型替你决定整套身份体系。

资料图 · 第 4 页 官方网页讲义第4/39页：需求工程：直面 Essential Complexity。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 4 · 原始讲义 · 相关讲解 00:05:44（不是同秒视频截图） · 官方网页讲义第4/39页：需求工程：直面 Essential Complexity。相关时间仅定位本节讲解，非同秒视频截图。

用户名密码看似是最自然的默认选项，却会引出注册、密码确认、忘记密码、验证身份、短信或邮件送达等**连串承诺**。第三方身份登录减少部分自建工作，但又引入接入条件、费用与组织资格等约束。它们都是需求和商业条件的选择，不只是代码怎么写。

> 案例回扣　登录链条：账号密码 → 忘记密码 → 身份确认 → 邮件/短信服务 → 服务配置与失败处理。评估时可以把每条新增承诺写出来，再比较第三方身份登录方案。

> 案例回扣　讲师把收集用户密码作为极端反例并指出不合规；笔记保留其说明“设计动机不同会改变方案”的作用，不将它写为产品建议。

深度优先做 MVP 与**广度优先**比较方案并不矛盾。前者尽快验证最重要的价值，发现需求是否值得继续；后者让关键决策有替代方案，避免一条路径走得很深后才发现成本不能接受。

需求最终需要被表达出来。代码可以反向还原出很多功能事实，但功能清单往往很长，还缺少设计宗旨：哪些复杂工作交给第三方，哪些能力必须自己掌握。架构讨论需要这些上层理由。

> **整理者说明**
> 讲义补充：需求越完备，歧义通常越少，但每条承诺都增加持续审核的工作，在 AI 工作流中还体现为上下文、token 与错误空间。具体登录供应商的当前政策并未在本笔记另行调查。

第 04 节　　00:10:51—00:14:22 · 课堂时间

架构的两项工作：划分领域规则，限制随意组合

> **关键判断 · 整理者提炼**
> 职责、数据归属和接口减少实现时的随意选择，但也形成需要长期承担的约束。

架构首先分解无法避免的**领域复杂性**，再减少实现附带的困难。把模块隔离并提供清楚的接口，使用者便能依赖其**契约**，而不必每次展开内部实现。数据库就是课堂反复使用的例子：应用通过接口管理逻辑数据，把存储、事务等机制交给下层。

架构设计

分解 Essential Complexity；减少 Accidental Complexity

- 好的架构能更容易地实现需求、承受需求的变更
  - 把容量、资格、时间冲突分开建模，让每一部分都能单独理解。
  - 让页面、批处理共用规则接口，减少重复实现与口径不一。

收缩 Spec / Impl Space 的可能性

- 明确模块职责、数据归属和接口约定，排除大量随意组合。
- 一个实用判断：**需求改一处，需要跟着改多少处?**

资料图 · 第 5 页 官方网页讲义第5/39页：架构设计。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 5 · 原始讲义 · 相关讲解 00:10:51（不是同秒视频截图） · 官方网页讲义第5/39页：架构设计。相关时间仅定位本节讲解，非同秒视频截图。

校园信息系统展示了另一类边界：统一身份认证有利于减少重复填写，但财务系统有记账、平衡、审计等自己的**领域规则**。系统之间的数据共享与通知需要共同接口，不能把“全部打通”理解为抹去各领域约束。

> **整理者说明**
> 讲义补充的实用检查：同一条课程容量规则应被学生选课、管理员补选和批量导入共同使用；需求改一处时，需要跟着改多少处，可以暴露规则是否散落。

需求收缩 Intent Space 的不确定性；架构收缩 Spec/Impl Space 的可能性。同一意图可能对应多种规约，同一规约又有多种实现。明确职责、**数据归属**与接口，就能减少模型或工程师临场猜测的部分，同时也承担相应的长期承诺。

读《Structured Programming》：AI 帮助进入原始经典

> **关键判断 · 整理者提炼**
>
> 经典阅读帮助理解抽象为何出现，层次化设计则回应人同时能处理的信息有限这一约束。

讲师现场打开 Dahl、Dijkstra、Hoare 的《**Structured Programming**》(1972)。书里的程序在今天看来很小，讨论顺序、选择、循环以及是否需要 GOTO；但小程序承载的是如何构造可理解系统的问题，而非过时的语法练习。

现场先让 AI 总结全书，再选择局部深入。讲师回顾自己整理系统、编程语言、软件工程经典文献的经历，强调**第一手文献**可以通过 AI 更容易地进入，而不是停在书名、脚注或二手评价。有关“黄金年代”的判断属于他的研究经验与偏好。

> **整理者说明**
>
> 课堂中关于具体 AI 产品语音、云电脑、额度的叙述是录制时的使用体验，本笔记不把它当作当前功能承诺。

阅读方法是从自己已知的知识往外扩：作者要解决什么问题，采用什么新手段，得到什么结论，最后怎样与已有知识连成网络。AI 可以帮助索引、翻译、对话和寻找类比；真正理解仍需**回到原文**与具体例子。

Dijkstra 的“小脑袋”观点解释了为什么需要架构：人一次能精确处理的元素有限，不能把理解十个状态的方法机械放大到一万个。**层次化分解**让我们逐层理解，尊重认知限制。

资料图 · 第 6 页 官方网页讲义第6/39页：On Our Inability to Do Much。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 6 · 原始讲义 · 相关讲解 00:14:22（不是同秒视频截图） · 官方网页讲义第6/39页：On Our Inability to Do Much。相关时间仅定位本节讲解，非同秒视频截图。

第 06 节

00:20:59—00:25:36 · 课堂时间

枚举、归纳、抽象：珍珠项链和协程的共同问题

> **关键判断 · 整理者提炼**
>
> 抽象应减少同时理解的细节，协程则让合作任务各自保留进度并遵守协作协议。

课堂从三种 mental aids 进入：**枚举**帮助检查有限情况，**归纳**帮助把重复过程建立在一个可复用的论证上，抽象帮助暂时忽略已被封装的细节。它们都在降低同时需要放进脑中的信息量。

Our Mental Aids

枚举、归纳、抽象

- Dijkstra 的 Part I 讨论了 our mental aids：**枚举、归纳、抽象**。
- 虽然讨论的案例在今天看来都是 “小程序”，但 necklace (pearls) 的比喻就是在分解复杂性。

Necklace (Pearls)：设计决策之间的依赖

- 每颗 pearl 承载一项独立设计决策；决定的先后，会改变其他层必须知道多少细节。
  - 过早把 image 解释成 lines，中间各层就都要理解 lines；推迟表示决策，更多层才能独立于它。原文：On What We Have Achieved

资料图 · 第 7 页 官方网页讲义第7/39页：Our Mental Aids。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 7 · 原始讲义 · 相关讲解 00:20:59（不是同秒视频截图） · 官方网页讲义第7/39页：Our Mental Aids。相关时间仅定位本节讲解，非同秒视频截图。

珍珠项链（necklace/pearls）把程序理解为可逐段掌握、相互衔接的设计。要关心的不仅是切成多少块，还有决定的顺序：一个过早确定的底层表示，会迫使后面的许多层知道它。好的接口让不同实现可以替换。

> **整理者说明**
> 讲义补充：image 过早变成 lines，会让中间层都背上“行”的表示细节；延迟该决定则能保持更多层独立。这里的原书页码与下方讲义截图汇编页码是不同的编号体系。

Hierarchical Program Structures

Small Head

- Part III: “precise thinking is possible only in terms of a small number of elements at a time...”
- SIMULA class/object abstraction

控制流解耦：Subroutine vs. Coroutine

- 原书的例子：让两个下棋程序互相对弈。谁应当是主程序，谁应当是子程序?
- 子程序服从 “调用—返回”；协程让双方保留各自的执行位置，交替恢复。Part III, §4, pp. 184–185

资料图 · 第 8 页 官方网页讲义第8/39页：Hierarchical Program Structures。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 8 · 原始讲义 · 相关讲解 00:20:59（不是同秒视频截图） · 官方网页讲义第8/39页：Hierarchical Program Structures。相关时间仅定位本节讲解，非同秒视频截图。

书中 SIMULA 的 class/object 把数据与操作打包成概念，**协程**则进一步处理控制流之间的合作。讲师以多人工作为类比：每个人记住自己的进度，在共享环境中通过协议协作；多个任务线不必硬塞进同一个巨大控制流程。

协程不仅有轻量执行的性能意义。回到早期设计问题，可以看到它也是一种复杂性分解手段：各个合作程序保存自己的执行位置，以合适的时机暂停、恢复。课堂提到下棋和编译器等场景。

> **整理者说明**
> 讲义补充：两个对弈程序都要“计算—步—交给对方—等待回应—继续”。强行规定一个为主程序、另一个为子程序会扭曲对等关系；**协程**保留双方结构。讲义将原书位置标为 Part III §4，印刷页184–185。

第 07 节 00:25:36—00:30:39 · 课堂时间

现场拆第七节，并把 DP 的状态含义放回代码

> **关键判断 · 整理者提炼**
>
> 把状态含义和迁移显式表达出来，可以先验证可理解的实现，再追求更高效的表示。

讲师让 AI 面向本科生解释 Part III 第七节 Concept Hierarchies。核心做法是把“**数据＋操作**”封装成一个概念，再逐层构造更接近问题领域的语言；在教务系统里，课程、教室与选课操作也应有合适的职责边界。

> **整理者说明**
>
> 讲义补充：原书第七节的线索为 TWLIST → MINISIM → Job Shop Model（印刷页208–220）。每层新增什么概念、上层因此不必关心什么，适合作为逐层阅读问题。该节未在课堂逐行读完。

资料图 · 第 9 页 官方网页讲义第9/39页：让 AI 带我们读第七章。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 9 · 原始讲义 · 相关讲解 00:25:36（不是同秒视频截图） · 官方网页讲义第9/39页：让 AI 带我们读第七章。相关时间仅定位本节讲解，非同秒视频截图。

视频原帧 现场 AI 导读 Concept Hierarchies；右侧显示概念层次说明，左侧保留课堂板书。

视频原帧 00:26:18 · 现场 AI 导读 Concept Hierarchies；右侧显示概念层次说明，左侧保留课堂板书。

接着用动态规划调试说明“小程序也有架构”。一两百行 DP 若只有 DP[i][j][k] 与三重循环，索引稍有错误可能不崩溃，却输出错误结果。难点是代码丢失了我们思考时本来存在的**状态语义**与**状态转移关系**。

解决思路是把状态抽象找回来：说明 i、j、k 各自代表什么，将一个 state 能转移到的后继显式列出，必要时打印或**可视化**。调试时观察的就不再是没有含义的数组格子，而是能对应优化问题的节点和边。

> 案例回扣 课堂示意 getDescendant(i,j,k)：返回此状态可能的迁移；进一步可以用有语义的 State 替代裸下标。目的在于能逐层检查状态与转移，而不只是把同一段循环换一个名字。

课堂以无后效性的有向无环状态图帮助理解这一例子。可以先使用易观察的状态对象和转移函数完成调试，再改写为等价且更高效的数组代码。这是在可理解性与执行效率之间分阶段做决定。

第 08 节

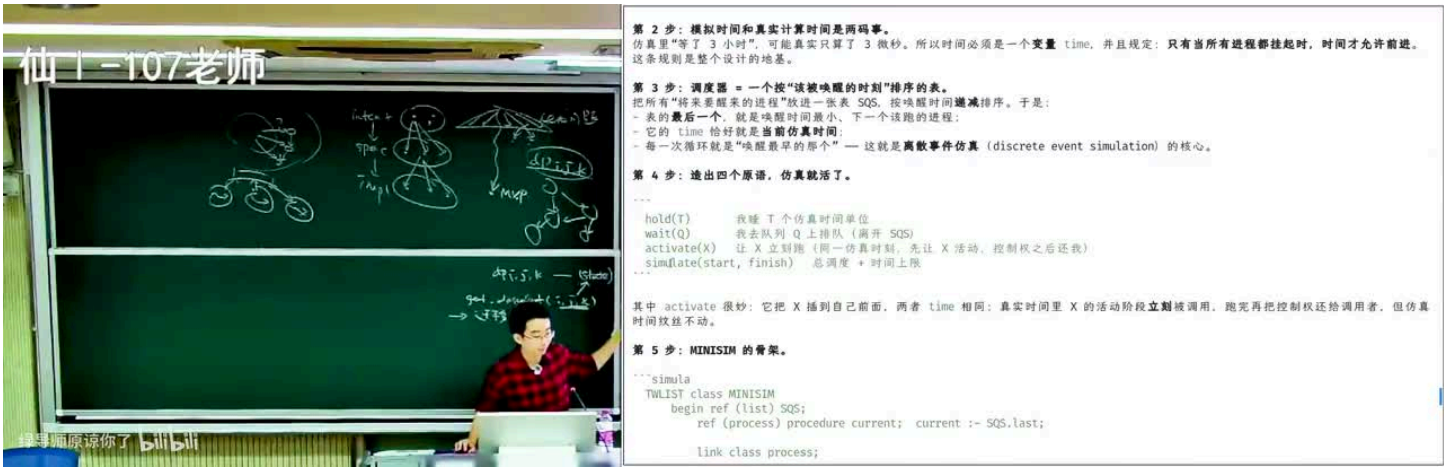
00:30:39—00:34:12 · 课堂时间

离散事件模拟与现代框架：类比要带着边界使用

> **关键判断 · 整理者提炼**
>
> 离散事件抽象分开模拟时间与运行耗时，跨系统类比仍需原文或实现证据来验证。

沿概念层次继续，课堂关注 hold、wait、activate、simulate 等原语。与直接写一个巨大状态机相比，合适的协程和时间抽象让程序表达“等待、推进**模拟时间**、唤醒另一项工作”，隐藏底层调度细节。

现场 AI 回答把**模拟时间**与真实计算耗时分开，并说明按**唤醒时间**排序的队列。模拟世界过了三小时，不代表程序必须运行三小时；系统应推进到下一个需要处理的事件，而非机械地等待现实时间流逝。



视频原帧 现场回答展开离散事件模拟：模拟时间、按唤醒时间排序的表，以及 hold/wait/activate/simulate 四项原语。

视频原帧 00:31:09 · 现场回答展开离散事件模拟：模拟时间、按唤醒时间排序的表，以及 hold/wait/activate/simulate 四项原语。

讲师继续要求 AI 举出现代前端的对应例子，得到 React Scheduler/Fiber 的**类比**，再进一步追问抽象层面最经典的案例，得到 UNIX、关系数据库与 Internet。由具体案例上升到共同问题，再回落到自己的系统，是这段演示的方法价值。

> **整理者说明**
>
> 整理者边界：视频中 React Scheduler/Fiber 与 MINISIM 的表格是现场模型生成的**类比**，未在本次逐项核对 React 源码，不把它写成机制完全等价或历史继承关系。

视频原帧 AI 生成的 MINISIM 与现代前端对照表；保留演示结果，同时标明这里只是待验证的类比。

视频原帧 00:32:34 · AI 生成的 MINISIM 与现代前端对照表；保留演示结果，同时标明这里只是待验证的类比。

讲师提醒：模型若没有检索，往往从预训练知识里找熟悉答案；若要比较大范围论文，应让它**实际检索**或从已索引的原文库中选择。这也是"听起来合理的**类比**"和"经原文支撑的比较"的差别。

第 09 节

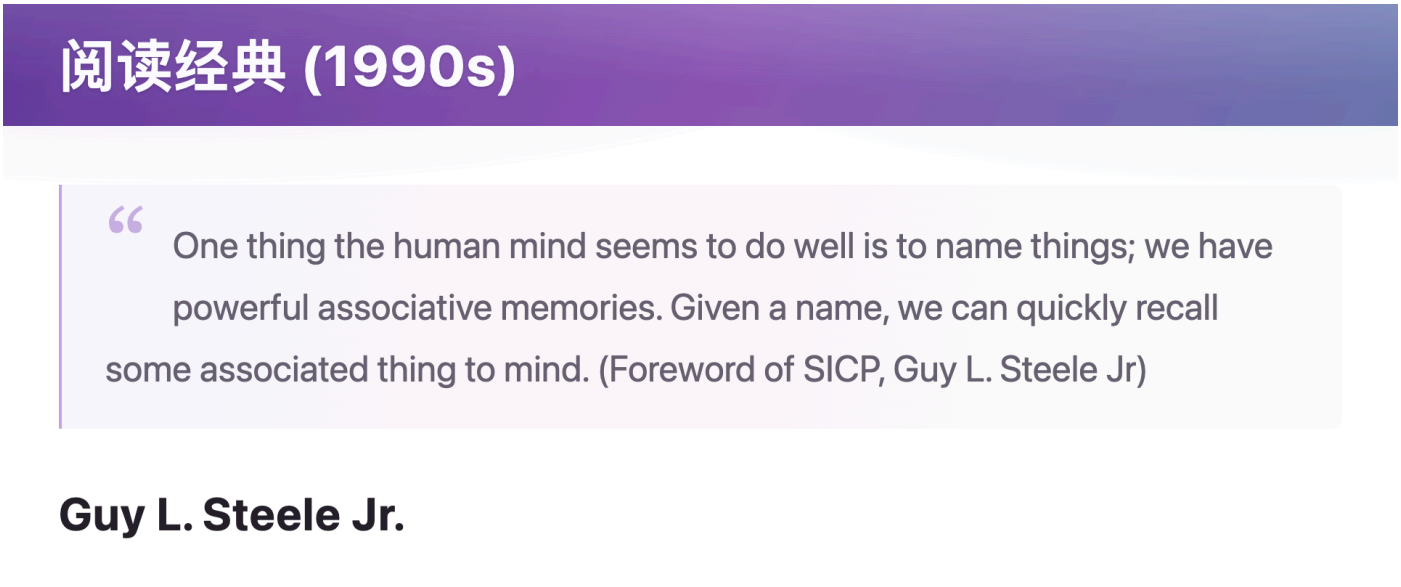
00:34:13—00:40:51 · 课堂时间

Growing a Language：让听众亲历一种语言的生长

> **关键判断 · 整理者提炼**

> 从受限词汇开始再引入定义，让听众体会表达能力为何需要可扩展的抽象。

第二份经典材料是 Guy L. Steele Jr. 在 OOPSLA 1998 的演讲 **Growing a Language**。讲师先介绍其 Scheme、Lisp 和 Java 语言规范背景，再现场打开文章，展示为什么只扫几眼标题和长段落容易错过关键设计。



资料图 · 第 10 页 官方网页讲义第10/39页：阅读经典 (1990s)。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 10 · 原始讲义 · 相关讲解 00:34:13（不是同秒视频截图） · 官方网页讲义第10/39页：阅读经典 (1990s)。相关时间仅定位本节讲解，非同秒视频截图。

这场演讲采用表演性的限制：从**单音节词**开始，多音节词要先定义，或由已经说明的规则构造。听众在理解内容之前，先亲身体会“词汇太小使表达复杂想法变得困难”，再看语言如何获得更强的表达能力。

Growing a Language

> It is good to design a thing, but it can be far better (and far harder) to design a pattern...We should now think of a language design as being a pattern for language designs, a tool for making more tools of the same kind.

让听众经历语言的“生长”

- 演讲从单音节词开始；多音节词要先定义，或用已说明的构词规则生成。听众亲身经历：先建立词汇，才能表达复杂思想。(原文，pp. 3–4)

使用模式：长程任务中积累的顶级人类智慧

- 保留 slots、允许以后选择和演化的 design space
- Steele 引用建筑师 Christopher Alexander 的《俄勒冈实验》：预先定死总体规划，会夺走使用者塑造未来的机会。**哪些选择留给后来的人，也是一项设计。** (原文，pp. 8–9)

资料图 · 第 11 页 官方网页讲义第11/39页：Growing a Language。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 11 · 原始讲义 · 相关讲解 00:34:13（不是同秒视频截图） · 官方网页讲义第11/39页：Growing a Language。相关时间仅定位本节讲解，非同秒视频截图。

编程语言也面对类似张力。它需要精确的**形式语义**，所以无法把日常世界的一切概念直接作为原语；但极小的原语集合虽然足以计算，不一定适合人类表达。Brainfuck 的 Hello World 在课堂中作为“可计算不等于好表达”的极端例子。

C 的 qsort 是更实际的例子：调用者需要处理比较函数指针、const void*、元素数量与元素大小。讲师借此批评抽象表达的负担，并不是在否认 C 的所有价值，而是让大家看见语言本身也有架构取舍。

> **整理者说明**
> 讲义补充：Steele 在 SICP 前言谈到**命名与联想记忆**；一个名字能让人重新唤起已经理解的概念。语言和模块设计的共同任务之一，是让复杂知识能由较少的概念承载。

从一个解到可生长的解空间

> **关键判断 · 整理者提炼**
>
> 好的架构提供可组合的扩展规则，使未来需求能在已有结构中增加、替换和组合。

小语言容易实现与移植，却可能难以表达不断增长的需求；一次设计庞大语言，又可能交付太慢，在成功前就被更小的方案取代。课堂将“**生长**”视为第三条路：提供足够好的**扩展原语**和组合机制，让后续使用者继续创造。

> **整理者说明**
>
> 讲义补充：Steele 借 Christopher Alexander《俄勒冈实验》讨论预先定死总体规划的问题；它提醒设计者把塑造未来的一部分选择留给使用者。这不是要求提前实现所有可能扩展。

语言的泛型、运算符机制与用户定义类型，都是在决定哪些抽象能由使用者建立。讲师以 C 到 C++ 为例指出：具备扩展能力并不保证结果优雅，模板既能构造强大抽象，也可能被用成极难理解的程序。这里保留的是他的设计评价。

架构因此不限于“选哪种数据库、定义 Student 与 Course 两个类”。二维码案例中设计一个小语言，也是在设计架构。共同问题是：未来出现新需求时，能否在现有规则中增加、替换和组合，而不是不断破坏旧结构。

好的架构不只是实现当前用例清单，而是定义开放需求空间对应的**解空间**。扩展机制应能参与组合；哪些约束固定、哪些槽位留给以后，本身就是设计决定。

资料图 · 第 12 页 官方网页讲义第12/39页：好的架构：设计解空间。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 12 · 原始讲义 · 相关讲解 00:40:51（不是同秒视频截图） · 官方网页讲义第12/39页：好的架构：设计解空间。相关时间仅定位本节讲解，非同秒视频截图。

> **案例回扣** OJ 的答案通过后常常即弃，而产品代码需要继续维护。讲师提醒不要把“本题过了”形成的习惯直接搬到长期软件：应把需求分解成领域概念及其组合，而非为每个用例独立打补丁。

第 11 节

00:46:56—00:49:19 · 课堂时间

阅读经典如何变成自己的设计判断

> **关键判断 · 整理者提炼**
>
> AI 可以降低进入经典的门槛，理解仍需连回已有知识并检验自己的设计选择。

讲师把经典阅读与**设计品味**联系起来：海量教程、零散经验和 AI 生成文本，容易让人只习惯于能运行的答案。经典的价值在于看见顶级工程师如何面对长期工作中的真实困难，理解为什么某种抽象值得保留。

Comment: 有太多宝藏被 “埋在” 这个快速发展的时代了

- 海量人类产物 (CSDN、防自学教科书……)、AI slop 在疯狂污染大家的品味
- 阅读经典是成长必备的道路
- 我真的很期待一个 “大学毁灭计划”
  - 所有学校的改进空间都实在太大了

把经典读进自己的设计

- 拿自己的系统追问：作者把哪些判断交还给了使用者？这些选择，需要什么共同规则才能彼此兼容？

13 / 39

资料图 · 第 13 页 官方网页讲义第13/39页：阅读经典，建立品味。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 13 · 原始讲义 · 相关讲解 00:46:56（不是同秒视频截图） · 官方网页讲义第13/39页：阅读经典，建立品味。相关时间仅定位本节讲解，非同秒视频截图。

初学者读得慢是正常的，因为需要先建立核心知识，才能把新文章连到**已有网络**。AI 的帮助是**降低查找、翻译和追问的门槛**，不是跳过理解。讲师期待把多年经典中的原始知识拆解后重新串联，形成面向个体的教材。

“大学毁灭计划”在课堂中是关于教育重构的倡议式表达：不要仅因教科书近、老师推荐或论文新，就放弃向更早的一手思想追问。读后最好回到自己的系统，问哪些**共同规则**能让后来者保有选择。

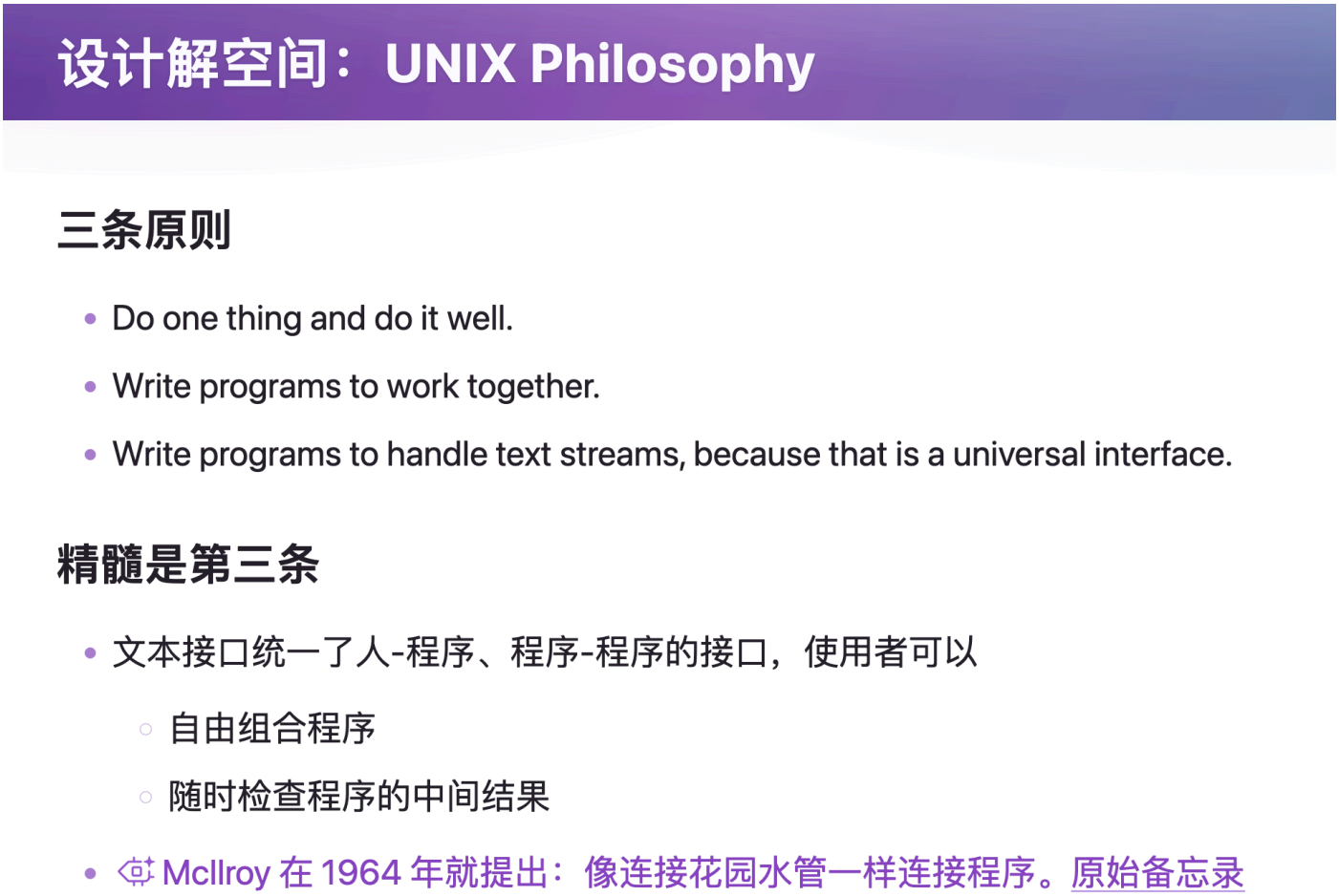
第 12 节

00:49:20—00:55:01 · 课堂时间

UNIX：文本接口把工具与人的选择串起来

> **关键判断 · 整理者提炼**
>
> 文本接口、输入输出和退出状态等共同约定，让专门工具和用户的选择组成新用途。

UNIX 的三条课堂原则是做好一件事、让程序能合作、用**文本流**作为通用接口。第三条让前两条获得实际的连接方式：同一份输出可以给人查看、保存为文件，或成为另一个程序的输入。中间结果可观察，也使调试更直接。



资料图 · 第 14 页 官方网页讲义第14/39页：设计解空间：UNIX Philosophy。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 14 · 原始讲义 · 相关讲解 00:49:20（不是同秒视频截图） · 官方网页讲义第14/39页：设计解空间：UNIX Philosophy。相关时间仅定位本节讲解，非同秒视频截图。

文本本身仍不够，工具还需要**共同语言**：大致一致的命令行参数、可处理的输出格式、退出状态与管道。已有 sed、awk 等工具的使用习惯，反过来促使新工具产生更容易被组合的输出。

> **整理者说明**
> 讲义补充：stdin/stdout/stderr 分离数据与诊断，管道默认连接 stdout 和 stdin；**退出码**按工具约定解读，grep 的1通常是未匹配，不能一概叫程序错误。

UNIX：组合需要共同约定

POSIX：命令与进程的公共接口

- 参数：短选项、选项参数、操作数；-- 结束选项解析（个别工具有历史例外）。语法约定
- 数据与诊断分开：stdin 输入、stdout 输出、stderr 诊断；| 默认连接前者的标准输出与后者的标准输入。Shell
- 退出状态单独传递：0 通常表示成功，其他值按工具约定解释；例如 grep 的 1 表示未匹配。退出状态

社区实践：让文本成为稳定的数据接口

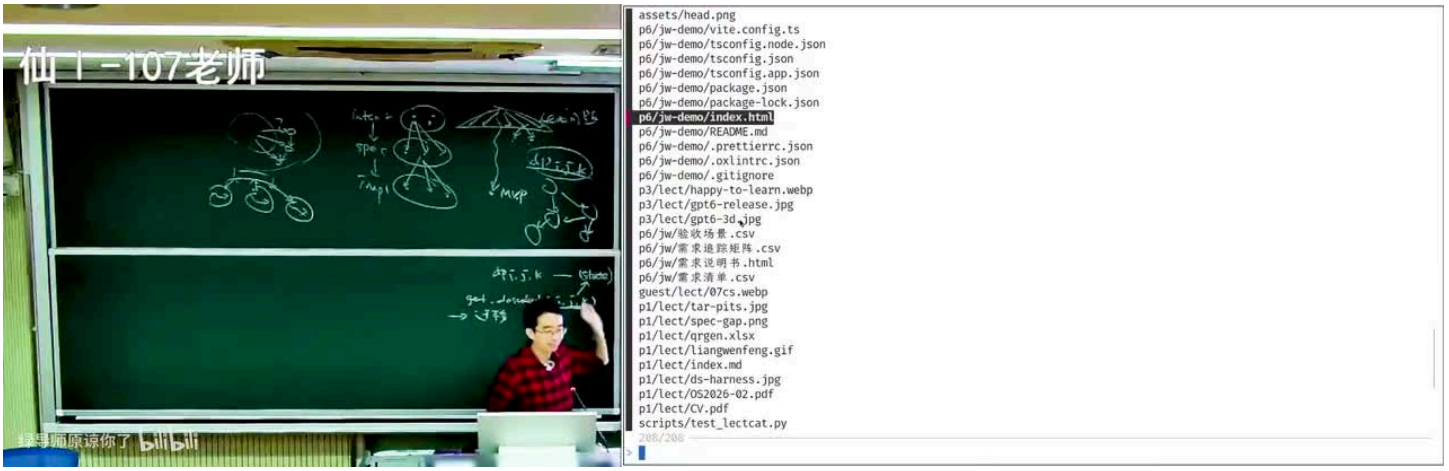
- 简单记录常用“一行一条”、固定字段；复杂结构用 TSV、JSON 等明确格式。
- 机器模式保持稳定；彩色、进度条、提示语不混入数据流。
- 分隔符也是协议：文件名可以包含换行，传文件列表可用 NUL 分隔。GNU 工具实践
- 排序、字符分类会受 locale 影响；需要可复现规则时可固定 LC_ALL=C。POSIX 环境变量

资料图 · 第 15 页 官方网页讲义第15/39页：UNIX：组合需要共同约定。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 15 · 原始讲义 · 相关讲解 00:49:20（不是同秒视频截图） · 官方网页讲义第15/39页：UNIX：组合需要共同约定。相关时间仅定位本节讲解，非同秒视频截图。

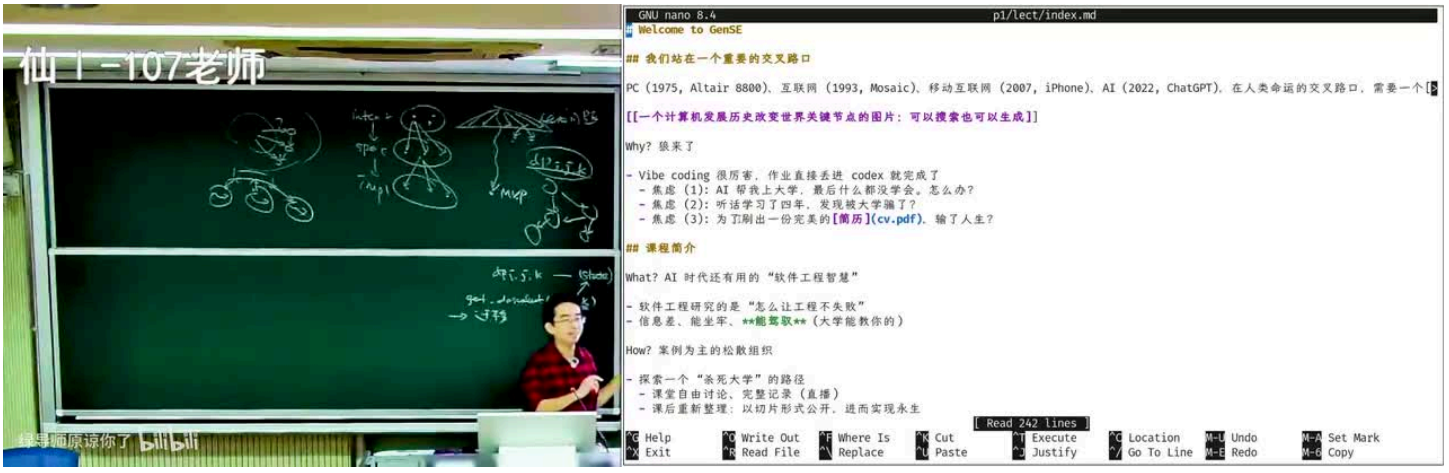
课堂比较了 GUI 的“打开文件”对话框：内置版本和系统版本都未必符合个人习惯，而且难以替换。随后现场为 nano 接入一个可交互选择文件的列表，选中第一讲的 index.md 后直接在编辑器里打开。选择器与编辑器并不需要由同一作者设计。

> 案例回扣 视频保留两个阶段：文件候选列表中的选择状态，以及 nano 成功打开 p1/lect/index.md 的结果。这样可看出工具组合确实完成了一次操作，而不只是一张菜单截图。



视频原帧 现场文件选择器：候选文件通过文本接口提供，用户可以选出要打开的文件。

视频原帧 00:53:52 · 现场文件选择器：候选文件通过文本接口提供，用户可以选出要打开的文件。



视频原帧 选择后的结果：GNU nano 打开 p1/lect/index.md。与前一图共同保存工具组合的完整演示。

视频原帧 00:54:14 · 选择后的结果：GNU nano 打开 p1/lect/index.md。与前一图共同保存工具组合的完整演示。

这一结果由 Shell 编程能力、**文本接口**与工具约定共同促成。用户输入一条命令时，也是在写一个小程序，把专门工具组合成自己的工作流。设计者给出的不是全部用途，而是创造用途的条件。

> **整理者说明**
>
> 讲义补充：机器输出应稳定，避免把颜色、进度条混入数据；复杂数据可用 TSV/JSON，文件名可能含换行时要考虑 NUL 分隔。jq→fzf 与窗口ID→fzf→xdotool 展示新的组合，xdotool 例子限定 X11。

UNIX：使用者创造新的用途

现代工具继续扩展组合的边界

- `jq`：把 JSON 中需要的字段提取成文本流，API 响应也能进入流水线。
- `fzf`：从标准输入读取候选项，让人选择，再把结果写回标准输出。
  - 书目 JSON → `jq` 提取书名 → `fzf` 人工选择：人的判断也是可组合的一步。
- `xdotool`：把窗口搜索、键盘和鼠标操作变成命令（适用于 X11）。
  - 窗口 ID 列表 → `fzf` 选择 → `xdotool windowactivate`：组合出自己的窗口切换器。

设计者提供了组合的规则，使用者可以创造设计者没有想到的用途

资料图 · 第 16 页 官方网页讲义第16/39页：UNIX：使用者创造新的用途。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 16 · 原始讲义 · 相关讲解 00:49:20（不是同秒视频截图） · 官方网页讲义第16/39页：UNIX：使用者创造新的用途。相关时间仅定位本节讲解，非同秒视频截图。

> **整理者说明**
> 复核修正：课堂约50:59口述“管道大概1983年引入UNIX”并不作为本笔记确定年代事实保留；本节只采用讲义明确给出的1964年 McIlroy 水管比喻，不依赖该口述年份。

第 13 节

00:55:02—00:58:27 · 课堂时间

关系数据库：逻辑关系与物理存储分离

> **关键判断 · 整理者提炼**
> 关系模型让应用表达逻辑数据，存储与执行策略可以在共同接口下独立变化。

课堂回到 Codd 1970 年的关系模型：把现实世界中的实体与关系投射到二维表及其操作上，让使用者不必随数据量、访问流量和**底层表示**变化而重写全部应用。这里隔离的是“数据意味着什么”与“数据怎样存放”。

案例二：关系数据库

> A relation model of data as a basis for protecting users of formatted data systems from the potentially disruptive changes in data representation caused by growth in the data bank and changes in traffic. (E. F. Codd, 1970)

把物理世界状态投射到关系数据库中

- 分离 “数据的理解” 与 “数据在机器中的组织”
- “谁借了哪本书” 保持不变，数据页、索引、访问路径可以改变。Codd 的数据独立性

在逻辑关系上构建应用

- SQL、查询优化器、事务实现使 CRUD 成为标准的软件架构 pattern
- 随后发展的关系数据库系统，把 “声明要什么” 落实为可选择的执行计划。System R (1976)

资料图 · 第 17 页 官方网页讲义第17/39页：案例二：关系数据库。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 17 · 原始讲义 · 相关讲解 00:55:02（不是同秒视频截图） · 官方网页讲义第17/39页：案例二：关系数据库。相关时间仅定位本节讲解，非同秒视频截图。

SQL 提供描述所需结果的语言，**查询优化器**选择执行方案，事务让相关读写作为一个整体推进。应用关注**逻辑关系**，底层系统管理存储和执行。这种分工让图书馆、学籍、财务和订单管理都能在共同基础上生长。

数据库的 schema 由应用定义，因此其设计者无需提前猜出全部业务场景。数据库在当前状态上还可提供 view：通过一个查询形成对世界的某种切片。系统给的是关系、约束和操作空间，使用者决定具体实体与问题。

> 案例回扣 讲义图用“读者S17借了馆藏B42”贯穿现实、逻辑与物理三层。按读者查借了什么，或按馆藏查被谁借走，问题不同但依赖同一借阅关系；底层可以从堆文件和索引换成分区和索引。

资料图 · 第 18 页 官方网页讲义第18/39页：关系数据库：现实、关系与存储。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 18 · 原始讲义 · 相关讲解 00:55:02（不是同秒视频截图） · 官方网页讲义第18/39页：关系数据库：现实、关系与存储。相关时间仅定位本节讲解，非同秒视频截图。

> **整理者说明**
>
> 整理者边界：课堂“近乎无限scale”是强调关系数据库的扩展能力，不是容量和性能无限的技术保证；实际能力取决于系统与负载。

第 14 节

00:58:29—01:03:39 · 课堂时间

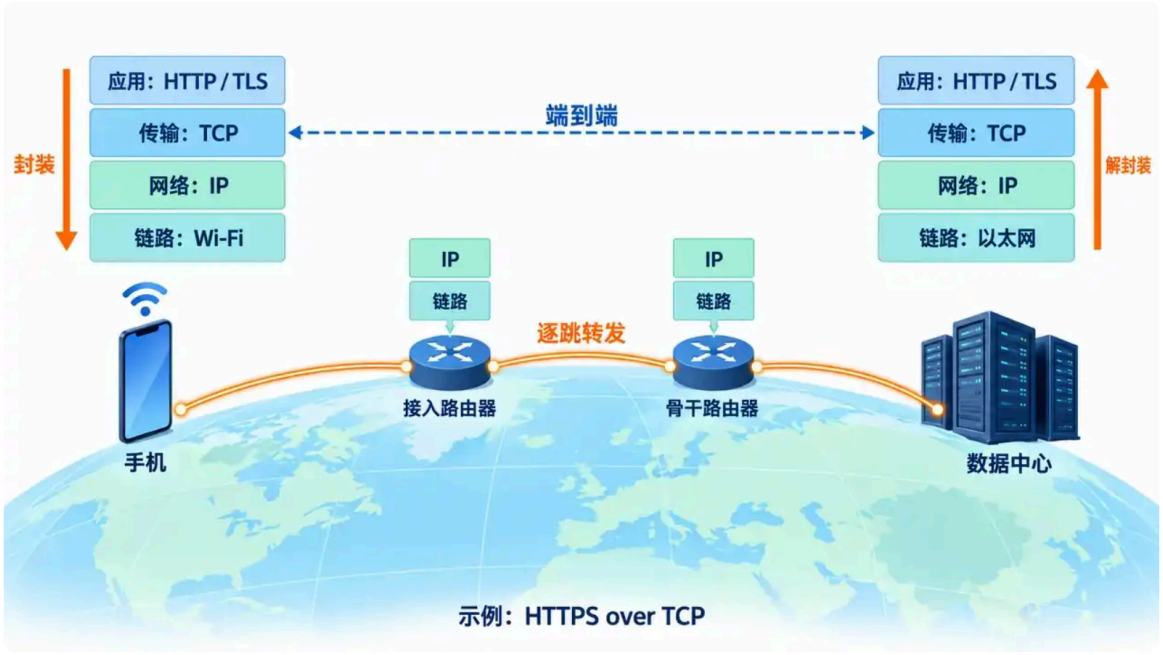
Internet：把长期变化留在共同协议之上

> **关键判断 · 整理者提炼**
>
> 共同协议划分网络责任，使上层用途持续演化，同时保留可靠性语义的适用边界。

Internet 的例子从课堂现场出发：电脑推流、手机监看和远端服务可以同时协作，因为基础网络提供了**通用连接能力**。上层不需要为每条物理链路或每次路径变化重写业务逻辑。

案例三：The Internet

手机与数据中心如何通信：TCP/IP 分层



资料图 · 第 19 页 官方网页讲义第19/39页：案例三：The Internet。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 19 · 原始讲义 · 相关讲解 00:58:29（不是同秒视频截图） · 官方网页讲义第19/39页：案例三：The Internet。相关时间仅定位本节讲解，非同秒视频截图。

讲师以 TCP 的**可靠传输**抽象说明分工：应用把数据交给传输层，而拥塞、连接中断等问题由协议机制处理；应用要表达什么则不由网络预设。可靠性是一组协议语义，并不等于连接永远不断。

> **整理者说明**
>
> 讲义补充图展示 HTTPS over TCP：应用/安全、TCP、IP、链路各层分别承担职责，两端处理与中间路由节点的职责不同。图示是特定协议组合，不代表所有 HTTPS 连接都只使用这一组合。

Internet：在持续变化中演化

RFC 1958: Architectural Principles of the Internet (1996)

> In searching for Internet architectural principles, we must remember that technical change is continuous in the information technology industry.

做好 “数据传输” 这一件事

- Telnet → ssh
- FTP/HTTP/... → HTTPS （许多常见用途的迁移）
- SMTP/IMAP 一直到今天还在使用

在共同规则之上，持续更换局部

- RFC 用城市作比喻：街道与建筑逐步更新，城市仍持续运转。
- 少量合作规则，可以生成庞大、多样、不断演化的技术空间。RFC 1958 §1

资料图 · 第 20 页 官方网页讲义第20/39页：Internet：在持续变化中演化。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 20 · 原始讲义 · 相关讲解 00:58:29（不是同秒视频截图） · 官方网页讲义第20/39页：Internet：在持续变化中演化。相关时间仅定位本节讲解，非同秒视频截图。

RFC 1958（1996）强调信息技术**持续变化**。架构应容纳用途、安全要求与实现方式的变化，而不是把未来定死。课堂回顾 Telnet、FTP 到 SSH、HTTPS 的常见场景迁移，以及 RESTful API 等上层约定的生长。

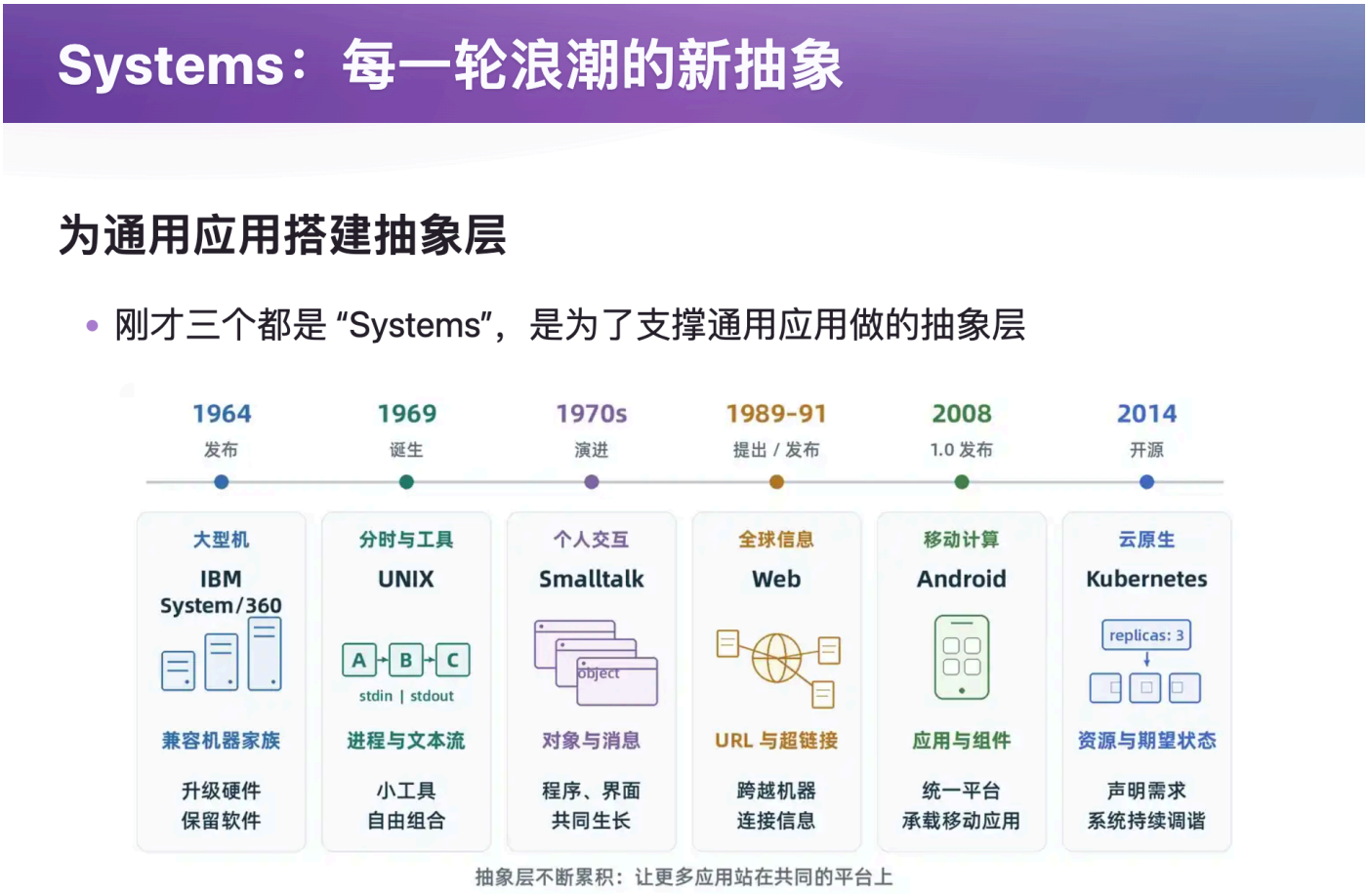
城市交通网络是本段比喻：共同通路使建筑可以局部更换，而城市继续运行。学习经典系统不仅为了手写 SQL 或了解协议名，更为了判断一种设计为什么能承载生态的**长期演化**。

第 15 节 01:03:39—01:07:00 · 课堂时间

从通用系统抽象到客户业务：一个请求、一段事务

> **关键判断 · 整理者提炼**
>
> 局部业务请求可映射为数据库操作，但一次事务不能替代跨多日的现实业务过程。

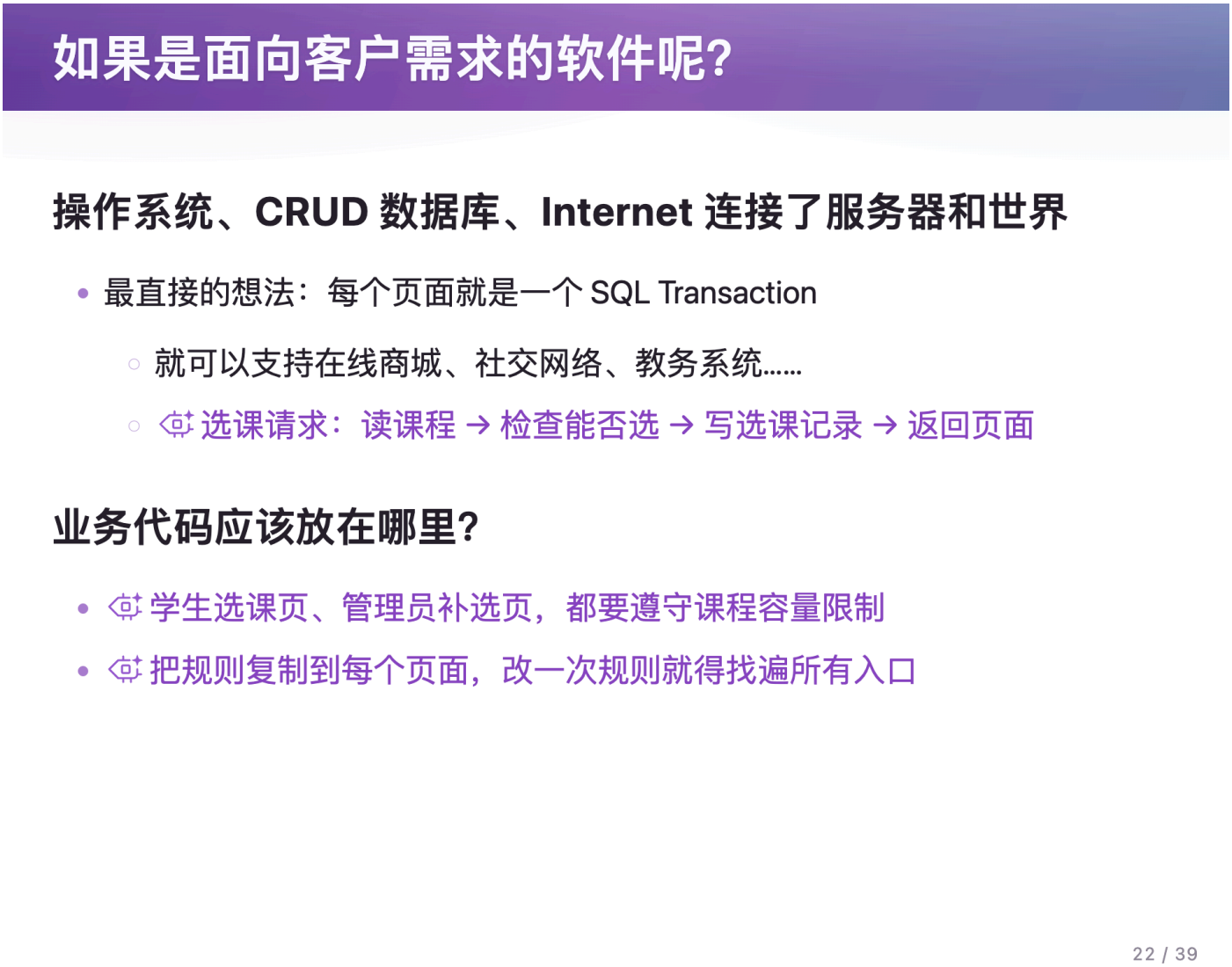
UNIX、数据库、Internet 都是支撑广泛应用的 Systems。讲师串起 IBM System/360、面向对象环境、移动平台和 Kubernetes，强调每轮浪潮都出现让应用站得更高的**新抽象**。



资料图 · 第 21 页 官方网页讲义第21/39页：Systems：每一轮浪潮的新抽象。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 21 · 原始讲义 · 相关讲解 01:03:39（不是同秒视频截图） · 官方网页讲义第21/39页：Systems：每一轮浪潮的新抽象。相关时间仅定位本节讲解，非同秒视频截图。

当操作系统、网络和数据库的基础具备后，业务软件最直接的组织方式是：把需求拆成**请求**，每个请求执行必要的**数据库操作**，再通过查询形成要展示的切片。它适合大量相对局部的信息管理功能。



资料图 · 第 22 页 官方网页讲义第22/39页：如果是面向客户需求的软件呢? 。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 22 · 原始讲义 · 相关讲解 01:03:39（不是同秒视频截图） · 官方网页讲义第22/39页：如果是面向客户需求的软件呢? 。相关时间仅定位本节讲解，非同秒视频截图。

这使“需求如何落成代码”不再神秘：选课**请求**读取课程、检查条件、写入选课记录；教师查看学生名单，则从选课关系中查询并呈现。应当作为整体保持一致的读写可放入**事务**。

> **整理者说明**
> 整理者边界：“每个页面就是一个SQL Transaction”是课堂用于理解早期业务应用的简化模型，不是所有网页都必须一页一**事务**，也不等于数据库事务可以涵盖跨数日的现实业务流程。

第 16 节

01:07:00—01:10:38 · 课堂时间

模板语言：让页面和 Prompt 保留本来的结构

> **关键判断 · 整理者提炼**
> 模板使固定结构与动态输入各有位置；语法变化本身并不会自动隔离业务规则。

Java Web 的普及依赖多种基础的汇合：运行时、自动内存管理、操作系统、网络与数据库。课堂用“Write once, configure everywhere”调侃部署带来的复杂性，说明抽象解决一种困难时也会带来新的取舍。

页面生成有两种方向：在 Java 中拼接大量 **HTML** 字符串，或者让 HTML 保持页面形状，在需要动态内容的位置嵌入代码。JSP 属于后一种写法，容器最终再把它翻译为 Servlet 执行。

资料图 · 第 23 页 官方网页讲义第23/39页：Java Web：直接在 HTML 里写代码。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 23 · 原始讲义 · 相关讲解 01:07:00（不是同秒视频截图） · 官方网页讲义第23/39页：Java Web：直接在 HTML 里写代码。相关时间仅定位本节讲解，非同秒视频截图。

模板还支持变量与循环：把查询得到的 list/map 传入模板，逐项渲染到页面。**HTML**/DOM 与 CSS 又把**结构与样式分开**，于是同一份信息可以获得许多展示方式。

> **整理者说明**
> 讲义补充区分 JSP 脚本片段、表达式与 JSTL/EL；将脚本换成标签并不会自动得到 MVC，**业务规则**仍可能散落在每个页面。

JSP：从 Java 片段到表达式与标签

同一张页面，两代写法

- <% ... %> / <%= ... %>：执行 Java / 输出表达式
- JSTL 1.0 (2002) 提供标准标签与 EL；JSP 2.0 (2003) 将 EL 纳入页面语言

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html><body>
    <% String msg = "Hello, JSP!"; %> <!-- Scriptlet -->
    <h1><%= msg %></h1>
    <ul>
        <c:forEach var="user" items="${users}"> <!-- JSTL + EL -->
            <li>${user.name}</li>
        </c:forEach>
    </ul>
</body></html>
```

- **换成标签，不会自动得到 MVC：业务规则仍需要独立的归属。**

24 / 39

资料图 · 第 24 页 官方网页讲义第24/39页：JSP：从 Java 片段到表达式与标签。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 24 · 原始讲义 · 相关讲解 01:07:00（不是同秒视频截图） · 官方网页讲义第24/39页：JSP：从 Java 片段到表达式与标签。相关时间仅定位本节讲解，非同秒视频截图。

讲师把这个经验迁移到 AI 应用的 Prompt：不要只用 A+B+C 拼接一串长字符串。可以把角色说明、instructions、日期等作为**模板变量**，使固定文字与动态输入各有清楚位置。

> 案例回扣 课堂示意：模板保留“you are …”等固定指令，在 instructions 位置填入运行时内容；使用 Jinja2 等模板语言可以将日期和其他变量交给渲染过程。重点是结构可维护，而非要求所有小Prompt都上模板框架。

第 17 节

01:10:38—01:12:48 · 课堂时间

MVC：状态、展示和输入分别由谁负责

> **关键判断 · 整理者提炼**
>
> Model 承担业务事实与规则，View 负责呈现，Controller 解释输入并协调操作。

MVC 将系统知道的事实与规则、事实的展示、用户输入的解释分开。课堂现场打开早期材料：**Model** 表达知识，**View** 是模型的视觉表示，Controller 连接用户与系统。

> **整理者说明**
> 复核口径：MVC 的早期交互式界面设计与后来的 Web MVC 并不完全相同。本节把课堂的 Web 说明保留为应用实例，不把 MVC 的起源写成1990年代数据库网页。

以教务系统理解，**Model** 包含课程、选课记录以及容量、时间冲突等业务规则；**View** 把业务结果呈现出来；Controller 接收请求、协调相应业务操作与展示。Model 不只是数据库里几张表的别名。

资料图 · 第 25 页 官方网页讲义第25/39页：MVC：让不同的变化各有归属。相关时间仅定位本节讲解，非同秒视频截图。

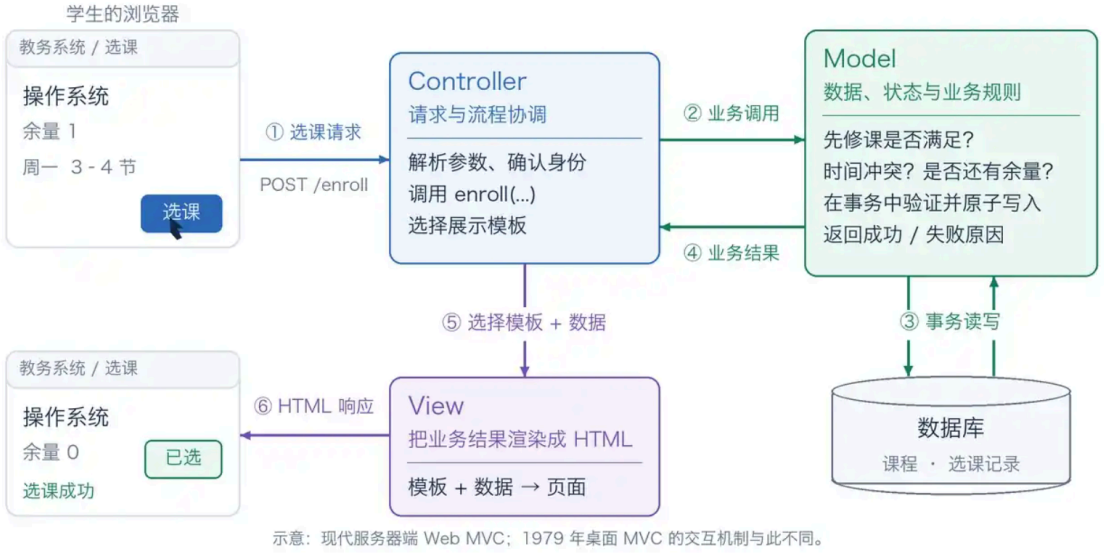
官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 25 · 原始讲义 · 相关讲解 01:10:38（不是同秒视频截图） · 官方网页讲义第25/39页：MVC：让不同的变化各有归属。相关时间仅定位本节讲解，非同秒视频截图。

讲师用 **View** 是 function of state 的思路强调展示的可推导性：显示结果本身不应顺手改变业务事实；改变事实应通过明确操作。在 Web 场景中，这通常体现为控制器组织请求、模型执行规则、视图渲染结果。

MVC：一次选课请求

Web MVC 中的业务流程与界面

- 请求处理：Controller 组织请求，Model 执行业务规则，View 呈现结果



示意：现代服务器端 Web MVC；1979 年桌面 MVC 的交互机制与此不同。

资料图 · 第 26 页 官方网页讲义第26/39页：MVC：一次选课请求。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 26 · 原始讲义 · 相关讲解 01:10:38（不是同秒视频截图） · 官方网页讲义第26/39页：MVC：一次选课请求。相关时间仅定位本节讲解，非同秒视频截图。

分离的收益是变化有了归属。课表从列表换成周历主要影响展示，选课限制变化主要影响**业务规则**；管理员补选等新入口应复用共同业务能力，并明确其不同授权，而不是复制一套似是而非的规则。

第 18 节

01:12:49—01:16:24 · 课堂时间

ViewState 也需要建模：为什么会走向 MVVM

> **关键判断 · 整理者提炼**
>
> 界面不仅依赖业务数据，还依赖可明确描述的 ViewState；派生关系让更新有据可循。

仅有业务数据库的状态，不能决定现代界面的全部样子：菜单是否展开、鼠标悬停在哪个元素、搜索框输入了什么、列表筛选后显示哪些行，都可能改变界面，却无需改动业务数据。

课堂抱怨的搜索体验是：整份列表已经在页面上，只想本地过滤，系统却仍要求提交搜索并刷新。原因之一，是把所有变化都当成数据库查询，而没有认真表达 View 自身的状态。

因此可以写出概念关系 View = render(Model, **ViewState**)。界面状态通常不必持久化，但不持久化不意味着可以不建模。把状态藏在分散的控件与事件回调里，仍会把同步负担交给程序员。

资料图 · 第 27 页 官方网页讲义第27/39页：把 ViewState 也当成 Model。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 27 · 原始讲义 · 相关讲解 01:12:49（不是同秒视频截图） · 官方网页讲义第27/39页：把 ViewState 也当成 Model。相关时间仅定位本节讲解，非同秒视频截图。

MVVM 将面向展示的状态与行为放到 **ViewModel**。课程列表和筛选词 query 是输入，visibleCourses 是**派生结果**；描述好依赖，框架就有机会自动维护界面，而不是每次事件后手工修改几个角落。

> **整理者说明**
> 讲义补充：Vue 可用 ref 保存query，用 computed 得到筛选列表；v-model 包装“状态→控件值”和“输入事件→状态”两条连接。**双向绑定**只说明这个绑定点，不表示全部组件数据可以任意双向流动。

用 Vue 理解 ViewModel

- Model：课程数据；View：筛选框与课程列表
- ViewModel：筛选词 `query`、派生列表 `visibleCourses` 及相关行为
- 用 `ref` 保存 `query`，用 `computed` 描述列表怎样依赖它

绑定关系写一次，变化由框架追踪

- `<input v-model="query">` 封装了两条连接：
  - 状态 → 控件：`:value="query"`
  - 输入 → 状态：`@input="query = $event.target.value"`
- 只保存 `query` 等必要状态；`visibleCourses` 从课程与筛选词派生，避免手工同步两份状态

资料图 · 第 28 页 官方网页讲义第28/39页：MVVM：让界面绑定状态。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 28 · 原始讲义 · 相关讲解 01:12:49（不是同秒视频截图） · 官方网页讲义第28/39页：MVVM：让界面绑定状态。相关时间仅定位本节讲解，非同秒视频截图。

第 19 节

01:16:26—01:19:20 · 课堂时间

React 与 Vue：隐藏更新机制，显式表达状态依赖

> **关键判断 · 整理者提炼**
> 界面从必要状态派生，框架负责更新机制；能计算的列表不必再独立维护一份状态。

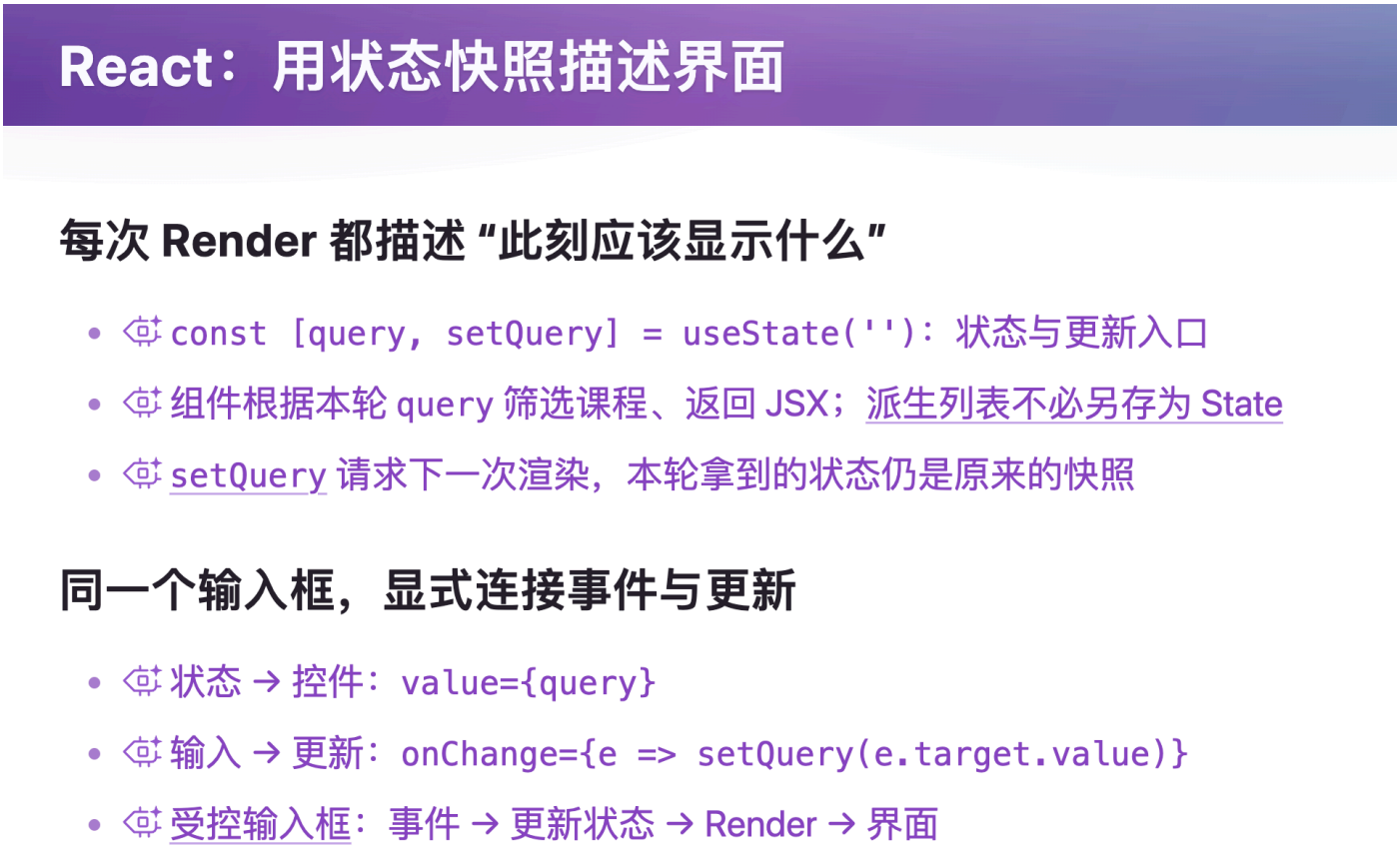
讲师提出一个理想：输入所有界面状态，用一个**无副作用**的函数算出 DOM 树。开发者可以先关心状态是什么、怎样变化，再关心确定状态如何呈现，不必同时手工安排每一步界面修改。

实际网页可能有大量元素，每输入一个字符都把所有内容重建会有性能问题。课堂用 Virtual DOM 等机制说明框架为什么承担更新优化：开发者遵循其**状态模型**，由框架组织实际界面更新。

> **整理者说明**
> 整理者边界：纯渲染是本节的设计原则，不表示真实应用没有副作用，也不表示所有更新仅靠Virtual DOM一种机制。

Vue 的讲解侧重响应式依赖关系，React 的讲解侧重按状态描述界面。讲师明确说自己对两者最新变化使用不多，因此不把“哪个更好”当成结论。共同目标是隔离复杂性，让状态来源、状态变化与展示逻辑更容易分别理解。

> 案例回扣 讲义具体化同一个筛选框：React 用 const [query,setQuery] = useState('') 保存必要状态；onChange 请求更新，下一轮渲染根据课程数据和query计算**可见列表**。既然列表可算，就没有必要再另存一份独立State。



资料图 · 第 29 页 官方网页讲义第29/39页：React：用状态快照描述界面。相关时间仅定位本节讲解，非同秒视频截图。

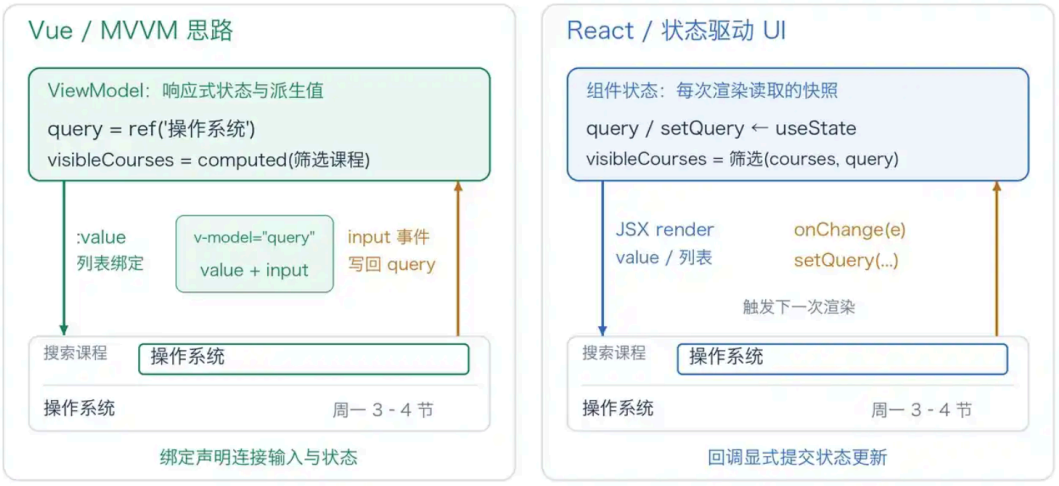
官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 29 · 原始讲义 · 相关讲解 01:16:26（不是同秒视频截图） · 官方网页讲义第29/39页：React：用状态快照描述界面。相关时间仅定位本节讲解，非同秒视频截图。

> **整理者说明**
> 讲义补充：setQuery 请求下一次渲染，本次渲染拿到的状态仍是**原快照**。Vue 的 Props 与 React 的组件数据流都从父到子；不要把框架差异简化为“Vue一切双向、React一切单向”。

Vue 与 React：同一个课程筛选框

都由状态决定界面，区别在如何表达更新

- Vue 用绑定语法连接输入与状态；React 在事件中显式调用状态更新函数



共同点：状态驱动界面，事件改变状态；Vue 组件间同样 props 向下、事件向上。

v-model 的双向连接针对绑定点；Vue 的组件 Props 与 React 的组件数据流都是从父到子。

30 / 39

资料图 · 第 30 页 官方网页讲义第30/39页：Vue 与 React：同一个课程筛选框。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 30 · 原始讲义 · 相关讲解 01:16:26（不是同秒视频截图） · 官方网页讲义第30/39页：Vue 与 React：同一个课程筛选框。相关时间仅定位本节讲解，非同秒视频截图。

第 20 节

01:19:20—01:23:07 · 课堂时间

长流程为什么击穿“放进 Model 就好了”的想法

> **关键判断 · 整理者提炼**
>
> 当前快照若同时承载过去、现在和未来，就容易丢失状态冲突的原因与决定依据。

MVC/CRUD 适合许多对象相对稳定、请求较局部的信息管理任务，但它没有消除**领域本质复杂性**。电商订单跨越下单、锁库存、优惠、支付、风控、发货、退款、售后和对账，任何环节失败都会影响其他环节。

应对复杂性

MVC/CRUD 解决了许多业务系统的架构问题

- 个人博客、社交网络、课程信息系统等相对简单的信息管理系统
- 特点：业务对象比较稳定、一次请求完成一小段局部操作

把复杂性放进 Model，不解决 Essential Complexity

- 长流程：下单 → 锁库存 → 优惠计算 → 支付 → 风控 → 发货 → 退款 → 售后 → 对账
- 强规则：修满学分 → 毕业设计 → 归还资产 → 特殊政策 → 毕业
- 会计账本：复式记账法

资料图 · 第 31 页 官方网页讲义第31/39页：应对复杂性。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 31 · 原始讲义 · 相关讲解 01:19:20（不是同秒视频截图） · 官方网页讲义第31/39页：应对复杂性。相关时间仅定位本节讲解，非同秒视频截图。

最直觉的实现是在 order 表里放 status，或者展开为是否下单、是否支付等若干列。流程越长，**状态组合**越多；一旦还允许返回、人工覆盖和特批，代码对各状态的解释更容易分叉。

课堂举“风控标为失败却已进入下一阶段”与“既能毕业又不能毕业”的状态作警示：错误不一定立刻崩溃，可能只是数据库里悄悄保存了**互相矛盾**的事实。直接手改某个字段暂时打通流程，却失去了为什么这样做的解释。

> **整理者说明**
>
> 整理者边界：讲师对学校系统数据状况的评价是课堂经验性推断，本笔记未审计学校数据库，不将其写成已核实的具体缺陷统计。

问题在于一个当前快照同时背着**过去、现在和未来**。过去怎样来到这里若已经丢失，后来看到冲突就很难还原原因。把复杂代码搬到Model目录，并不会自动解决这种时间与依赖的纠缠。

复式记账：校验结构与业务真实性是两回事

> **关键判断 · 整理者提炼**
> 复式记账用结构校验部分错漏，业务分类、遗漏与重复仍需核对凭证。

讲师从个人记账需求进入一个更容易观察的一致性问题。复式记账要求同一**业务**在相应账户上成对记录，**借贷合计平衡**，使一些错漏可以通过试算平衡发现。它不是只在余额上加减一个数字。

> **案例回扣** 讲义图：公司向银行借入10,000元，银行存款增加10,000，短期借款也增加10,000，资产=负债+所有者权益仍成立。“借/贷”是记账方向，不能直接等同于日常意义的收入/支出。

资料图 · 第 32 页 官方网页讲义第32/39页：复式记账：同一笔业务，保持会计等式。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 32 · 原始讲义 · 相关讲解 01:23:08（不是同秒视频截图） · 官方网页讲义第32/39页：复式记账：同一笔业务，保持会计等式。相关时间仅定位本节讲解，非同秒视频截图。

> **整理者说明**
> 复核补充：平衡只能检出部分错误。分类错了、遗漏了整笔**业务**，或者重复记入一组平衡分录，也可能仍然平衡。讲师“相信校验”的个人实践不应被写为自动保证账务正确；分录仍要核对**凭证**与业务。

正式报表包含现金、存货、固定资产、应付、借款与所有者权益等多个相关项目。一次**业务**可能同时影响多个位置；让人或模型直接填各格，很容易产生口径不一致。需要先表达这是什么业务，再按规则得到报表变化。

讲师没有把自己定位为会计专家，而是展示面对陌生领域怎样设计软件：当前状态可以保存，也可以保存状态怎样一步步形成。由此引入 Event Sourcing，把历史**业务**与**状态迁移**保留下来。

第 22 节

01:26:30—01:29:49 · 课堂时间

LLM as a Compiler：把原始材料编成可检查的动作

> **关键判断 · 整理者提炼**
>
> 模型解释材料，受约束动作与确定程序计算结果，这几层都需要分别检查。

这套记账架构先定义一组**受约束**的操作：每项记账动作明确规定资产负债表、利润表与现金流量表各自怎样变化。课堂展示的是“给定一份业务动作，会产生哪些确定影响”，而不是让语言模型随意修改所有报表格子。

LLM 充当**编译器前端**：读取持久历史里的原始业务材料，将其翻译成这些小操作。随后程序按时间顺序执行或reduce全部操作，形成当前报表。历史事实、模型的业务解释、**确定执行规则**因而有了分开的检查位置。

记账架构：Event Sourcing & LLM as a Compiler

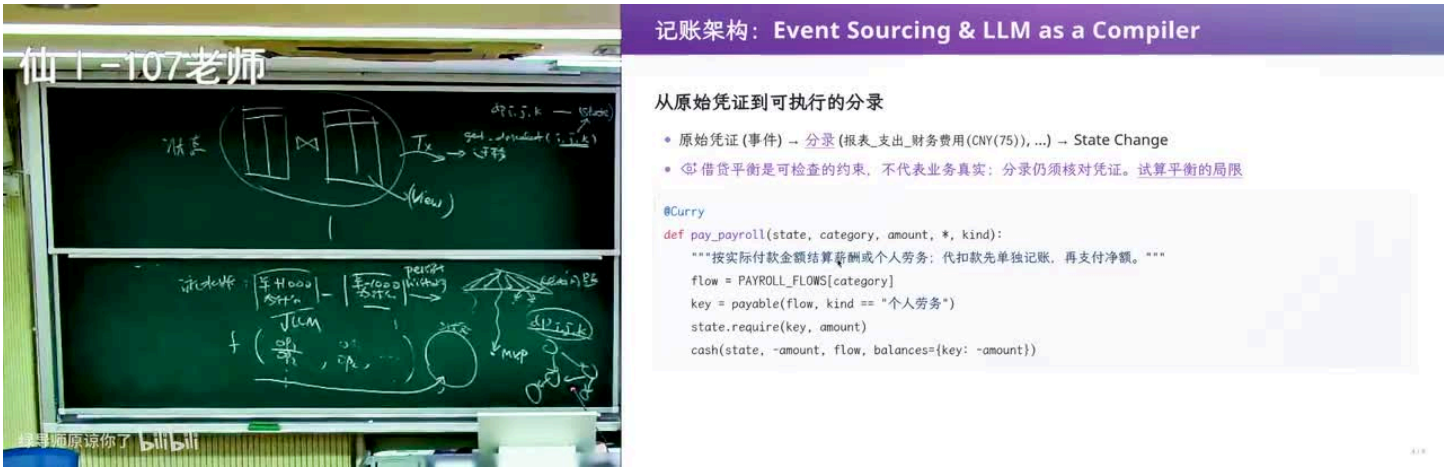
从原始凭证到可执行的分录

- 原始凭证 (事件) → 分录 (报表_支出_财务费用(CNY(75)), ...) → State Change
- 借贷平衡是可检查的约束，不代表业务真实；分录仍须核对凭证。试算平衡的局限

```
@Curry
def pay_payroll(state, category, amount, *, kind):
    """按实际付款金额结算薪酬或个人劳务；代扣款先单独记账，再支付净额。"""
    flow = PAYROLL_FLOWS[category]
    key = payable(flow, kind == "个人劳务")
    state.require(key, amount)
    cash(state, -amount, flow, balances={key: -amount})
```


资料图 · 第 33 页 官方网页讲义第33/39页：记账架构：Event Sourcing & LLM as a Compiler。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 33 · 原始讲义 · 相关讲解 01:26:30（不是同秒视频截图） · 官方网页讲义第33/39页：记账架构：Event Sourcing & LLM as a Compiler。相关时间仅定位本节讲解，非同秒视频截图。



视频原帧 板书完整展开“原始历史 → LLM编译的小操作 → 回放计算状态”；右侧同时显示官方记账操作代码。

视频原帧 01:28:58 · 板书完整展开“原始历史 → LLM编译的小操作 → 回放计算状态”；右侧同时显示官方记账操作代码。

课堂以特批毕业类比持久历史：若允许某个本不满足条件的学生毕业，应留下这项决定的记录，再让计算使用它，而不是只把数据库里的毕业标记改为1。例外也需要成为可解释的输入。

> **整理者说明**
>
> 复核边界：这些数字和pay_payroll内部语义由当前官方讲义补足，课堂口述侧重总体架构与currying。模型编译的动作和规则自身都可能错，仍需人工审计；本笔记不是该工具的会计合规认证。

> **案例回扣** 讲义中的银行收支例子包含财务费用75元、利息收入22.07元；货币资金净变化22.07−75=−52.93元。各报表相应变化都从两项动作与规则算出，而非让模型独立填格。本次用Decimal复核了该算式。

资料图 · 第 34 页 官方网页讲义第34/39页：从分录到报表。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 34 · 原始讲义 · 相关讲解 01:26:30（不是同秒视频截图） · 官方网页讲义第34/39页：从分录到报表。相关时间仅定位本节讲解，非同秒视频截图。

资料图 · 第 34 页 第34页表格的内嵌原始图，放大保留两项分录与三张报表变化。75元费用、22.07元利息，净变化−52.93元；左端凭证名称在源图中已遮挡。

官方网页讲义第34页内嵌原图；同页细节 · PDF物理页 34 · 原始讲义 · 相关讲解 01:29:20（不是同秒视频截图） · 第34页表格的内嵌原始图，放大保留两项分录与三张报表变化。75元费用、22.07元利息，净变化−52.93元；左端凭证名称在源图中已遮挡。

代码层的小架构也会影响可维护性。讲师用 **currying** 合并模型最初写得很长的重复代码，让部分参数先固定，再生成可复用的记账函数。大架构解决历史与状态，小抽象解决操作内部的重复。

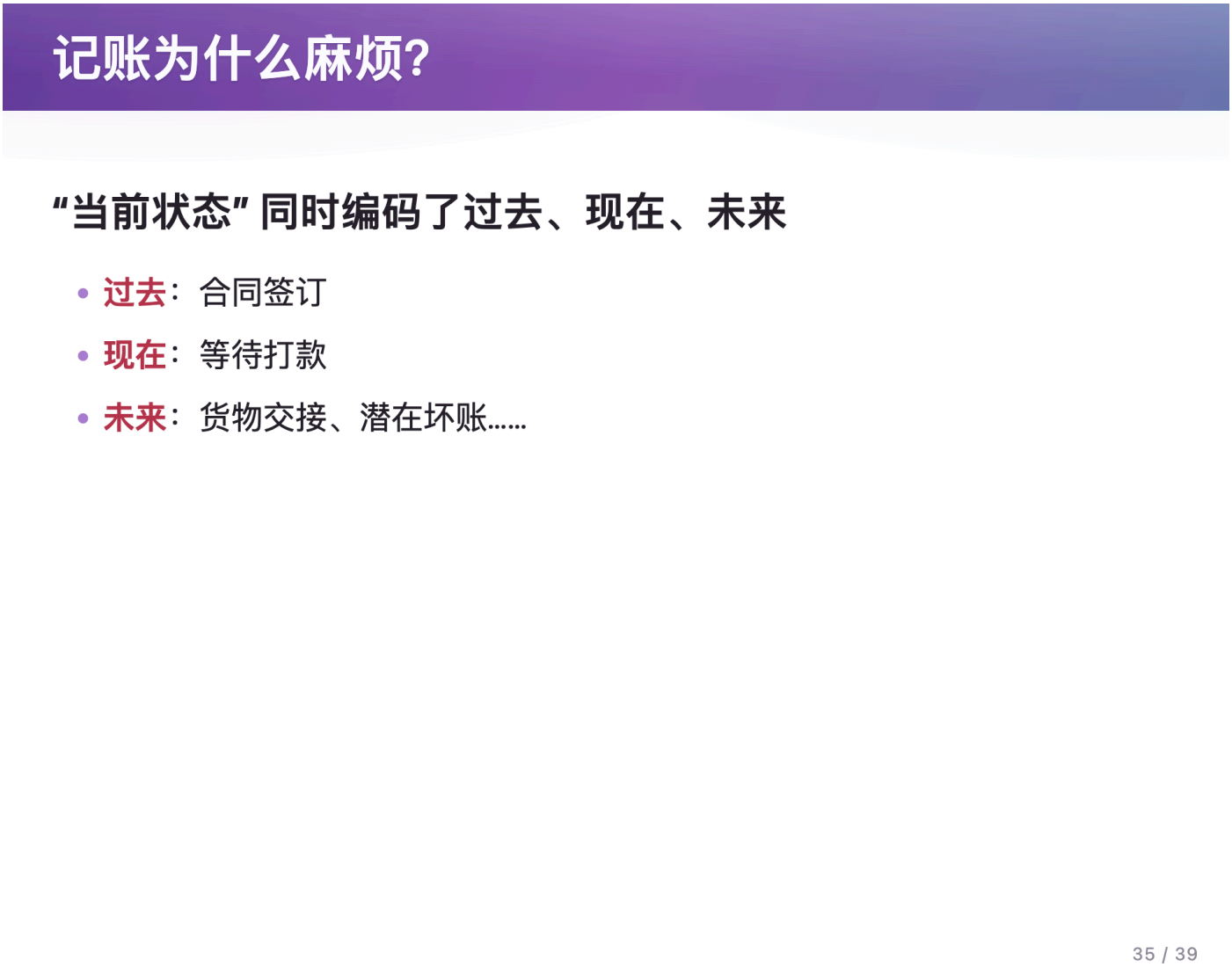
> 案例回扣　pay_payroll片段先按类别找到应付款项，state.require检查余额，再由cash同时减少现金与相应应付余额。代扣款另行记账后支付净额，区分确认费用、形成应付、实际付款，避免重复计费。该片段依赖项目金额类型和账本定义，不能单独当成完整程序运行。

第 23 节　　01:29:49—01:34:32 · 课堂时间

成绩更正的连锁反应：显式依赖让结果可重算

> **关键判断 · 整理者提炼**
> 成绩更正会影响多个业务判断，显式依赖和修订依据让受影响结果可以追溯与重算。

若只保存当前状态，规则变化或录入错误常常要求迁移、补丁和手工修数据。问题不是改一个字段困难，而是很难确认所有**依赖**它的结果是否同时更新。



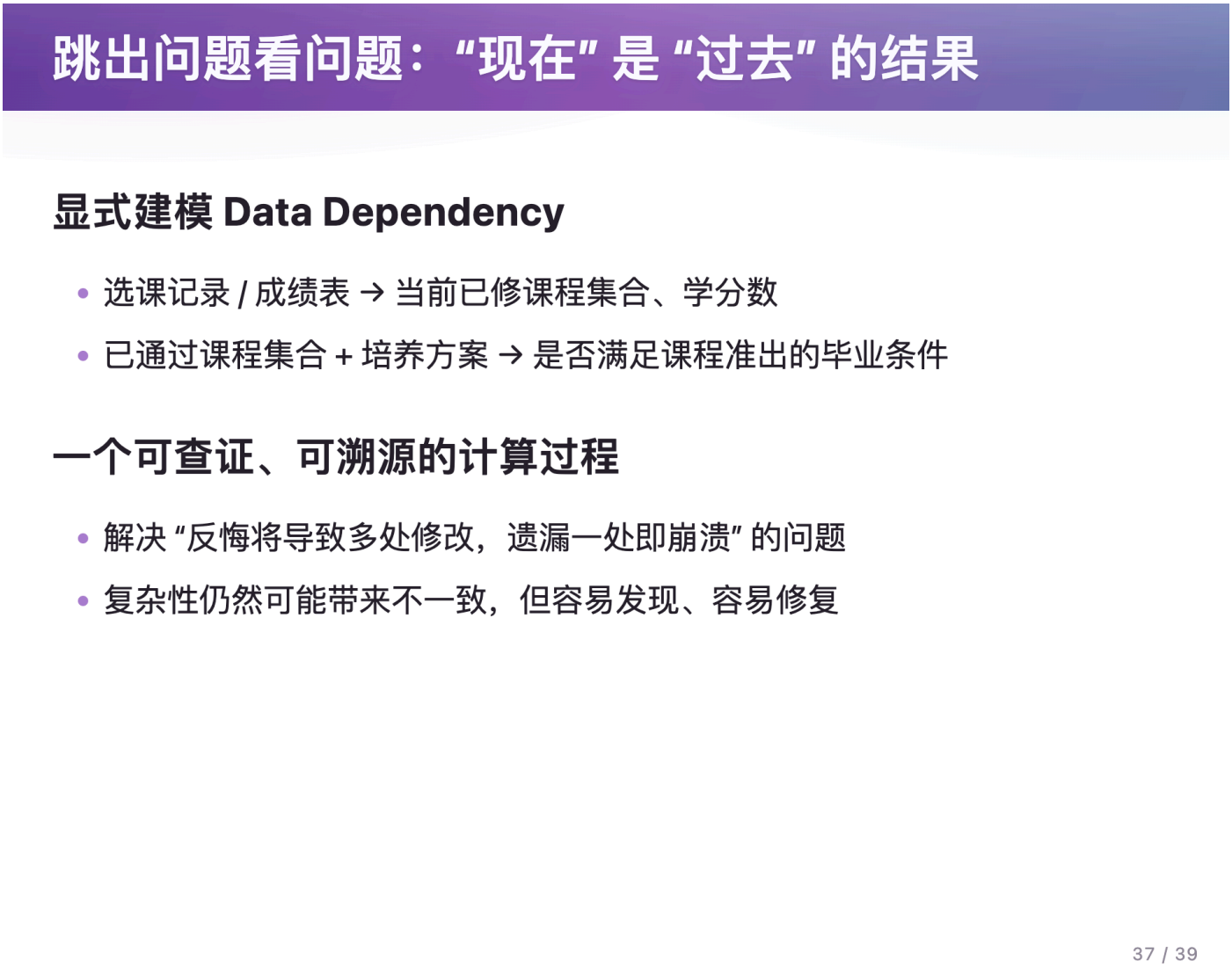
资料图 · 第 35 页 官方网页讲义第35/39页：记账为什么麻烦？。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 35 · 原始讲义 · 相关讲解 01:29:49（不是同秒视频截图） · 官方网页讲义第35/39页：记账为什么麻烦？。相关时间仅定位本节讲解，非同秒视频截图。

课堂的具体情境是**成绩复核**：一门原先通过的课改为未通过，会影响学分、毕业准出、下一门课的先修资格。如果学生已据此选上操作系统，再把计算机系统基础的通过标记改为0，系统要如何解释现有选课记录？原始**依赖**并不会因为数据库只改一格就消失。

> **整理者说明**
> 讲义补充：课程准出条件只是毕业条件的一部分，毕业设计、资产归还和批准事项应有各自明确的判断；不能把课程条件满足悄悄等同于全部毕业要求满足。

Event Sourcing 的价值在于保留事实和从事实到结论的计算路径。先明确成绩、课程学分、培养方案与资格判断怎样**依赖**，再在事实更正后**重算**受影响结果；即使没有单独保存一张依赖图，计算逻辑也应显式体现依赖。



资料图 · 第 37 页 官方网页讲义第37/39页：跳出问题看问题：“现在”是“过去”的结果。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 37 · 原始讲义 · 相关讲解 01:29:49（不是同秒视频截图） · 官方网页讲义第37/39页：跳出问题看问题：“现在”是“过去”的结果。相关时间仅定位本节讲解，非同秒视频截图。

模型把凭证翻译成操作也可能错误。把这一层与状态投影分开后，可以审查并更正解释，再重新计算报表。纠错不是神奇地没有代价，而是不用靠到处手改失去来源的数值来猜测影响范围。

> **整理者说明**
> 整理者边界：追溯事实与纠正解释要保留**修订依据**。对已确认历史，通常应记录更正、撤销或冲销等新事实，不能把“修改翻译结果”理解为可以无痕改写原始历史。

第 24 节

01:34:33—01:38:04 · 课堂时间

过去是事实，现在是视图，未来是预期

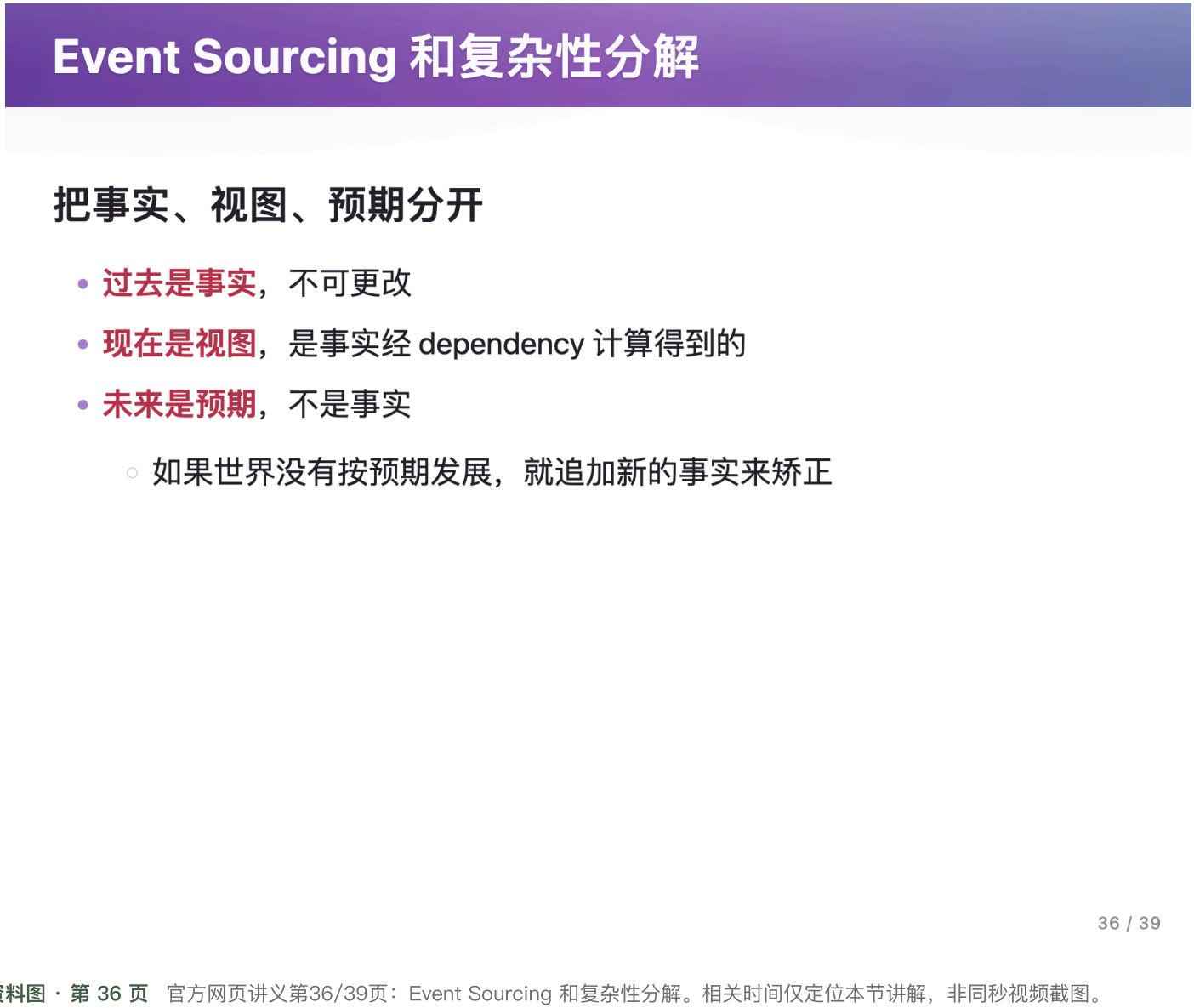
> **关键判断 · 整理者提炼**

> 过去事实、当前视图与未来预期应分开，重算还要保留相应时间的规则。

可回放计算不仅帮助纠错，也支持**情景推演**。以订单为例，支付之后若风控成功会到什么状态，失败又如何演化，可以在明确的假设下分别计算。记账系统同样可考察有多少应收无法回收会危及现金状态。

但架构只提供推演能力，概率来自额外模型和输入。不能因为用了Event Sourcing，就自动知道企业破产的概率；也不能把未来可能发生的款项当成已经发生的现金**事实**。

课堂把时间分成三部分：过去是已经发生的**事实**；现在是事实经规则计算的**视图**；未来是预期。世界没有按预期发展，就追加新的事实修正理解，而非让工程师悄悄改当前数据库，积累越来越难解释的矛盾。



资料图 · 第 36 页 官方网页讲义第36/39页：Event Sourcing 和复杂性分解。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 36 · 原始讲义 · 相关讲解 01:34:33（不是同秒视频截图） · 官方网页讲义第36/39页：Event Sourcing 和复杂性分解。相关时间仅定位本节讲解，非同秒视频截图。

一个获批准的特例同样要被记录。重新审核时，系统应知道是依据正常规则通过，还是依据哪项批准覆盖了哪条条件。保留决定来源，例外才不会成为下一轮计算时无法解释的漏洞。

计算还要知道当时的规则。课程学分可能变化，所以“通过ICS得到多少学分”需要取相应时刻的课程信息，而不能用今天的学分替代过去。这样的结果才具有查证与溯源意义。

第 25 节

01:38:04—01:39:41 · 课堂时间

SQL 的调侃与本讲收束：把一致性问题转成计算问题

> **关键判断 · 整理者提炼**
>
> 以可追溯事实为基础派生其他状态，把多处同步转为可测试的计算，同时保留重放边界。

最后，讲师借用“Billion Dollar Mistake”的说法对关系数据库提出挑衅式批评：它没有充分统一管理跨数据、跨应用的任意**计算依赖**。数据库里一个“能否毕业”的0/1字段，若来自复杂计算却被当作独立事实维护，就成为潜在不一致点。

38 / 39

资料图 · 第 38 页 官方网页讲义第38/39页：SQL 的 Billion Dollar Mistake 😄。相关时间仅定位本节讲解，非同秒视频截图。

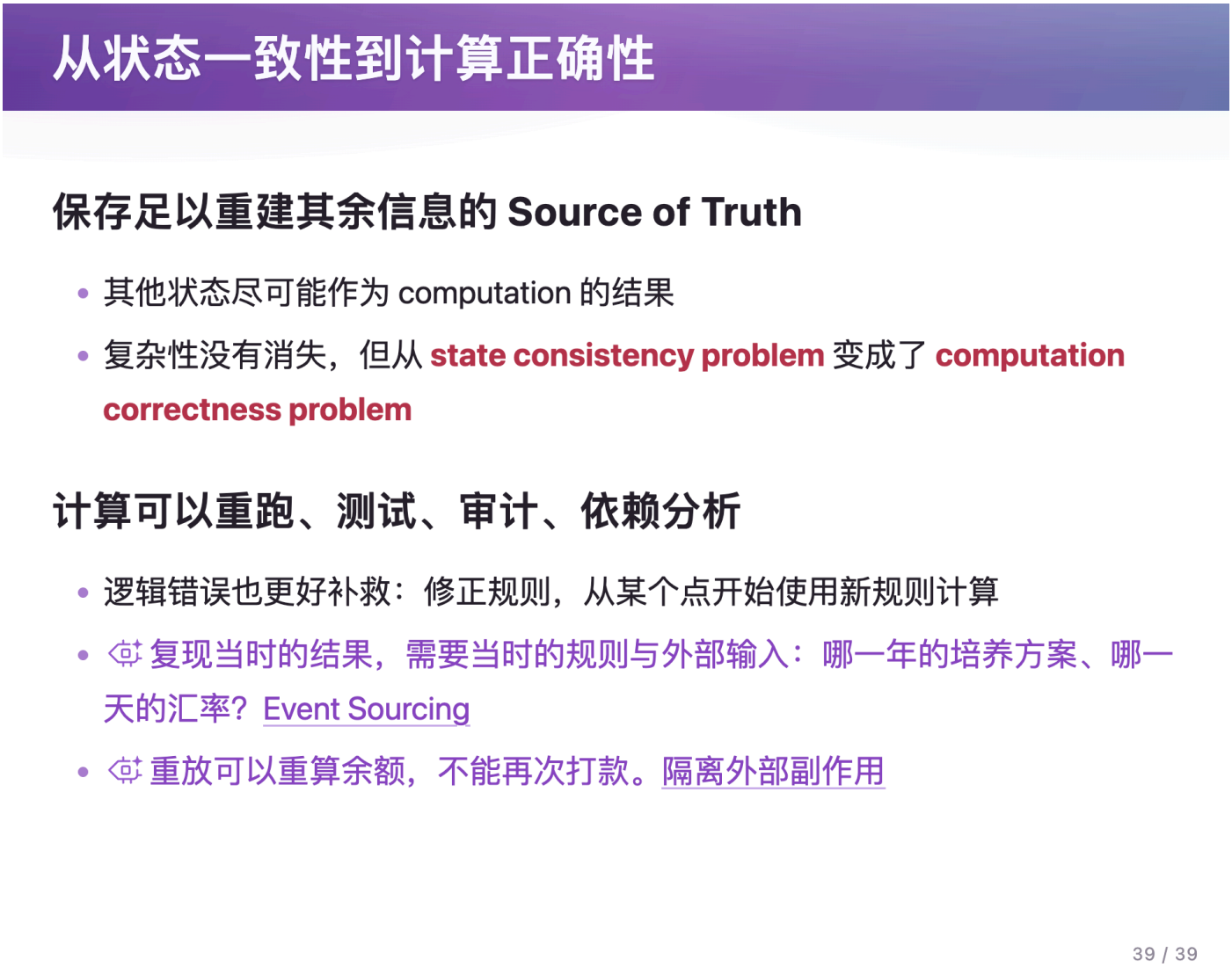
官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 38 · 原始讲义 · 相关讲解 01:38:04（不是同秒视频截图） · 官方网页讲义第38/39页：SQL 的 Billion Dollar Mistake 😄。相关时间仅定位本节讲解，非同秒视频截图。

这个批评不是说SQL没有视图或约束，也不是说关系模型不能表达计算。课堂关注的是基础记录变了以后，数据库、应用逻辑、缓存和界面里依赖它的结果由谁统一维护，怎样判断失效，怎样追溯来源。

MVVM、React、Dataflow、Event Sourcing 在不同层次把**派生状态**或依赖显式化：界面从状态派生，当前状态从历史派生。将它们放在一起，是一种看架构的视角，并非声称它们拥有同一种机制或同一个历史起因。

> **整理者说明**
> 讲义补充的收束：保留足以重建其余信息的**Source of Truth**，其他状态尽量作为计算结果。性能需要的缓存、索引、汇总可以保存，但须知道其来源、失效条件与重建方式。

> **整理者说明**
> 讲义补充：复杂性没有消失，而是从维护多份状态始终一致，转向检查计算是否正确。计算可重跑、测试、审计和做依赖分析；复现旧结果要保留旧规则和**外部输入**，重放余额更不能再次执行实际打款。



资料图 · 第 39 页 官方网页讲义第39/39页：从状态一致性到计算正确性。相关时间仅定位本节讲解，非同秒视频截图。

官方网页讲义截图汇编（本次生成，非官方原生PDF） · PDF物理页 39 · 原始讲义 · 相关讲解 01:38:04（不是同秒视频截图） · 官方网页讲义第39/39页：从状态一致性到计算正确性。相关时间仅定位本节讲解，非同秒视频截图。

> **整理者说明**
> 整理者练习：检查自己的AI应用，列出哪些是原始事实、哪些是人工决定、哪些只是派生结果。若一个字段变化需要手工想起五个同步位置，

> 就寻找能把依赖显式写出的边界。

自测：先回答，再核对

先用自己的话回答，再展开参考答案；答不清楚时，回到对应知识点。

问题 01

同样是选课系统，什么困难属于本质复杂性，什么属于附带复杂性?

参考答案 · 核对思路

容量、先修与时间冲突属于领域约束；重复数据、规则复制和手动缓存同步多由实现方式引入。架构分解前者，减少后者。

回查第 2 节 →

问题 02

为什么“支持登录”不算足够明确的需求?

参考答案 · 核对思路

还未决定身份来源、注册与找回方式、接入条件与费用；每个默认选择都会引出后续承诺。

回查第 3 节 →

问题 03

怎样让难调试的DP重新变得可观察?

参考答案 · 核对思路

恢复状态含义、显式列出转移，打印或**可视化**节点与边；先验证清晰实现，再改写为等价的高效表示。

回查第 7 节 →

问题 04

UNIX为什么不只是许多小工具的集合?

参考答案 · 核对思路

共同**文本接口**、输入输出约定、退出状态和管道使工具可组合，中间结果可检查，人的选择也能成为一步。

回查第 12 节 →

问题 05

MVC的Model是不是数据库表?

参考答案 · 核对思路

不只是表，还包括状态与业务规则；**View**负责呈现，**Controller**解释输入并协调业务。

回查第 17 节 →

问题 06

为什么不另存visibleCourses作为独立状态?

参考答案 · 核对思路

它可由课程数据与query派生，独立维护会增加同步与不一致机会。

回查第 19 节 →

问题 07

LLM做记账编译器，确定性代码与人工复核分别承担什么?

参考答案 · 核对思路

模型把凭证解释为**受约束**动作；程序确定执行动作并计算报表；复核检查凭证、解释、规则是否符合业务。平衡不保证真实性。

回查第 22 节 →

问题 08

成绩更正为什么不该只修改pass字段?

参考答案 · 核对思路

通过课程、学分、先修资格与毕业判断都可能**依赖**它。需要记录更正与规则，再**重算**受影响结果并保留来源。

回查第 23 节 →

问题 09

能够回放是否意味着可以无风险重做全部操作?

参考答案 · 核对思路

不能。回放应重建内部状态，不能重复实际付款等外部副作用；复现旧结果还需旧规则和当时的**外部输入**。

回查第 25 节 →